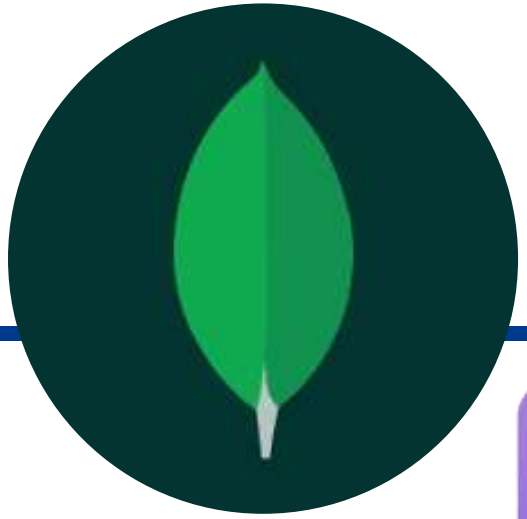
MONGODB, C# .NET







Anas Ben Yaiche

Droits réservés à Cap Compétences, ne pas reproduire

MongoDB

- Objectif :
  - Évaluer NoSQL
  - Comparer SGBD Relationnel vs MongoDB
  - Configurer MongoDB
  - Installer driver .NET
  - Manipuler objets & données
  - Implémenter une application
  - Requêtes avec pipeline d'agrégation

- Prérequis :
  - Bonnes connaissances en programmation C#

Sommaire



Introduction à MongoDB



Connexion et utilisation de MongoDB



Intégration de MongoDB avec .NET



Modélisation et indexation



Introduction à la réplication et au Sharding



Extension de MongoDB



Requêtes avancées et agrégations



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

INTRODUCTION À MONGODB

Module 1

Introduction à MongoDB

1. NoSQL : solutions et caractéristiques
2. Introduction au SGBD NoSQL MongoDB
3. La modélisation sous MongoDB
4. Les différents formats utilisés par MongoDB : JSON, BSON
5. Installation de MongoDB
6. Les outils MongoDB



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

NoSQL : solutions et caractéristiques

Chapitre 1

Historique des bases de données

- Modèle hiérarchique (1960s) **IMS (IBM)**

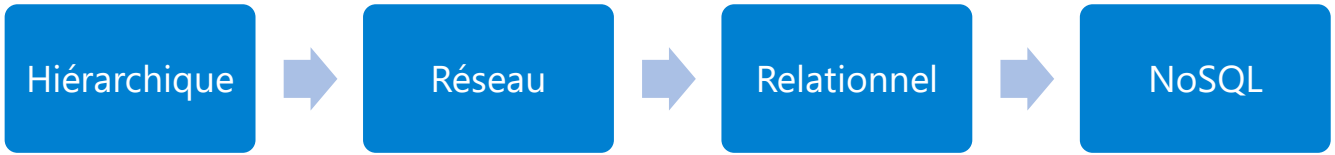
Les données sont organisées en arborescence (parent → enfant).

- Modèle réseau (1970s) **CODASYL DBTG**

Les données sont reliées par des graphes permettant des relations multiples.

- Modèle relationnel (1980s) **Oracle, MySQL, PostgreSQL**

Les données sont organisées en tables (lignes/colonnes) et manipulées avec SQL.



Le mouvement NoSQL

- Fin des années 90, les bases relationnelles sont les plus répandues.
- Les grandes entreprises du web ont été confrontées aux limitations des SGBD relationnels en termes de scalabilité
- Elles ont développé leurs propres systèmes de bases de données (Google : BigTable, Facebook : Cassandra, …)
- Fin des années 2000, apparition de plusieurs projets open-sources pour répondre à ces besoins.
- En 2009, le terme « NoSQL » apparaît comme intitulé d'une rencontre d'informaticiens à San Francisco.

Définition

NoSQL = « Not Only SQL »

Famille de systèmes de gestion de bases de données qui s'écartent du modèle relationnel traditionnel. Ils sont conçus pour gérer de grands volumes de données non structurées ou semi-structurées, avec une haute disponibilité et une scalabilité horizontale.

Schéma flexible

Pas de schéma fixe imposé. Les documents d'une même collection peuvent avoir des structures différentes.

Scale horizontal

Distribué sur plusieurs serveurs (nœuds). Ajouter des machines plutôt que d'en acheter une plus puissante.

Hautes performances

Lecture/écriture optimisées pour de gros volumes. Pas de jointures coûteuses comme en SQL.

Modèles variés

Documents (JSON), clé-valeur, colonnes larges, graphes. Chaque modèle répond à un besoin précis.

Les 4 familles NoSQL

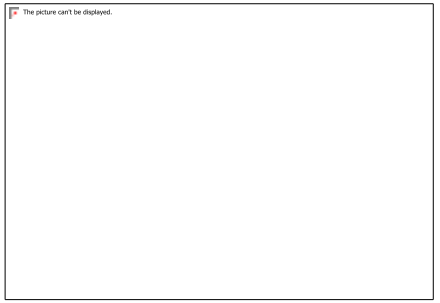
- Clé-Valeur

simple, rapide (Redis, DynamoDB)



- Colonne

orientée Big Data (Cassandra, HBase)



- Graphe

relations complexes (Neo4j, OrientDB)



- Document

flexible, JSON/BSON (MongoDB, CouchDB)

Les 4 familles de bases de données NoSQL

Type de base NoSQL	Structure de stockage	Cas d'usage typiques	Exemples connus
Clé–Valeur	Paires clé/valeur simples	Cache, sessions, profils utilisateur	Redis, Riak, DynamoDB
Orientée Colonne	Colonnes au lieu de lignes	Big Data, séries temporelles, analytics	Cassandra, HBase, ScyllaDB
Orientée Document	Documents JSON/BSON semi-structurés	Applications web, CMS, IoT, API	MongoDB, CouchDB, Firestore
Orientée Graphe	Nœuds et relations (graphes orientés)	Réseaux sociaux, recommandations, routes	Neo4j, ArangoDB, JanusGraph

Le théorème CAP

Théorème CAP

Dans un système distribué, il est impossible de garantir simultanément les 3 propriétés suivantes.

Au maximum, on ne peut en garantir que 2 sur 3 à la fois.



ACID et BASE

ACID

(Bases relationnelles)

Atomicité (A)

Tout ou rien : une transaction réussit entièrement ou échoue complètement.

Cohérence (C)

Les données respectent toujours les règles de validité définies.

Isolation (I)

Les transactions concurrentes ne s'interfèrent pas.

Durabilité (D)

Une transaction validée est persistée même après un crash.

BASE

(Bases NoSQL distribuées)

Basically Available (BA)

Le système répond toujours à chaque requête — les données peuvent être périmées.

Soft State (S)

L'état peut changer même sans nouvelle écriture (propagation asynchrone).

Eventually Consistent (E)

Les nœuds convergent vers le même état… avec un délai acceptable.

ACID et BASE

Critère	ACID	BASE
Signification	Atomicité, Cohérence, Isolation, Durabilité	Basically Available, Soft State, Eventually Consistent
Cohérence	Stricte et immédiate (toutes les lectures voient la dernière écriture)	Éventuelle (convergence différée entre les nœuds)
Transactions	ACID complètes (COMMIT / ROLLBACK automatique)	Limitées ou absentes (MongoDB 4+ : multi-doc ACID)
Scalabilité	Verticale (scale-up) Serveur plus puissant	Horizontale (scale-out) Ajout de nœuds cloud facilement
Disponibilité	Haute cohérence (au détriment de la disponibilité si partition)	Haute disponibilité (répond toujours, même données périmées)



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Introduction au SGBD NoSQL MongoDB

Chapitre 2

MongoDB

- Base de données orientée document
- Open-source
- Développé en C++
- Interactions en JSON, et données stockées sous forme BSON (JSON binaire)
- Schéma flexible au global et au sein d'une même collection
- Shell JavaScript pour l'accès aux données et l'administration

MongoDB : les avantages

- Conçu dans une optique de performance
- Prévu nativement pour être scalable (réplication et sharding)
- Nombreuses fonctionnalités (comparaison, logique, filtre de type, expression régulière, recherche texte, espace géographique, …)
- Permet également de stocker des fichiers de grande taille en grand nombre: « GridFs »
- Indexation des documents pour optimiser les performances
- Administration simple

MongoDB : cas d'utilisation et clients

- Adapté aux sites web à fort trafic et/ou à forte volumétrie de données
- 4.0 : Support des transactions (replication)
- 4.2 : Support des transactions distribuées (sharding)
- 5.0 : Ajout des collections de series de temps (Time Series)
- 5.3 : Ajout des collections « clustered »

- **Single View**
- **Internet of Things**
- **Mobile**
- **Personalization**
- **Catalog**
- **Realtime Analytics**
- **Content management**
- **Mainframe Offloading**
- **Gaming**

SQL vs NoSQL


SQL
(Relational Database)
NoSQL
(Not Only SQL)
Modèle de données
Basé sur des tables (lignes et colonnes)
Schéma fixe et prédéfini
Modèle de données
Modèles variés : documents, clés-valeurs, colonnes larges, graphes
Schéma flexible ou dynamique
Exemple de données
ID
Nom
Email
Âge
1
Alice
alice@mail.com
25
2
Bob
bob@mail.com
30
Exemple de données
{
"_id": 1,
"nom": "Alice",
"email": "alice@mail.com",
"age": 25,
"adresses": ["Paris", "Lyon"]
}
Langage de requête
SQL (Structured Query Language)
Langage
Langages de requête spécifiques (souvent inspirés de JSON)
Relations
Relations via clés primaires / étrangères (Jointures)
Relations
Peu ou pas de relations (données souvent dénormalisées)
Transactions
Transactions ACID complètes (Atomicité, Cohérence, Isolation, Durabilité)
Transactions
Transactions ACID partielles (selon le système)
Priorité à la disponibilité et à la performance
Évolutivité
Évolutivité verticale (ajout de ressources sur une machine)
Évolutivité
Évolutivité horizontale (ajout de machines/nœuds facilement)
Cas d'utilisation
Applications ayant des données structurées, transactions complexes et fortes contraintes de cohérence.
Ex : ERP, CRM, systèmes bancaires
Cas d'utilisation
Applications nécessitant une grande scalabilité, des données semi-structurées ou non structurées, et une forte disponibilité.
Ex : Big Data, IoT, réseaux sociaux, catalogues
Exemples de bases
MySQL
PostgreSQL
SQL Server
ORACLE
Exemples
MongoDB
Cassandra
Redis
DynamoDB
CouchDB



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

La modélisation sous MongoDB

Chapitre 3

Structures des données

- Un serveur MongoDB peut contenir plusieurs **bases de données**
- Chaque base de données peut contenir plusieurs **collections**
- Une collection est une série de **documents** identifiés par une clé unique
- Un document est composé de **propriétés** ou **champs** contenant des :
  - **Valeurs simples** (nombres, chaines de caractères, booléen, …)
  - **Objets** ou **sous-documents** contenant leurs propres propriétés
  - **Tableaux** de valeurs simples ou de sous-documents

La modélisation sous MongoDB

- MongoDB n’impose pas de schéma fixe pour les données au sein d’une collection.
- Chaque document peut être inséré dans une collection avec une structure différente.
- En général, les documents d’une collection partagent une structure similaire : la flexibilité facilitant les évolutions fonctionnelles.
- Le défi : équilibrer besoins applicatifs, performances du moteur et modèles d'accès aux données.
- Concevoir selon l'usage réel : requêtes, mises à jour, traitements — et selon la structure naturelle des données.



CAP COMPÉTENCES
Activateur de Talents

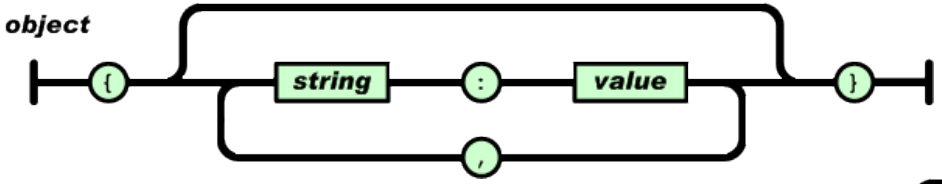
EURO INFORMATION

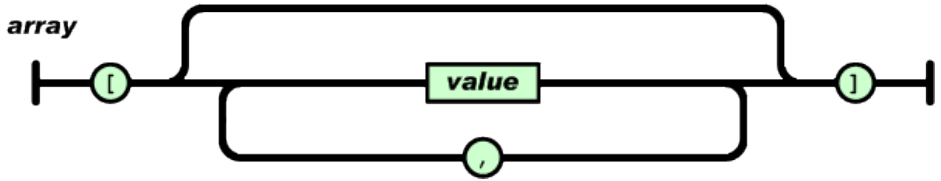
Les formats de MongoDB : JSON et BSON

Chapitre 4

JSON (JavaScript Object Notation)

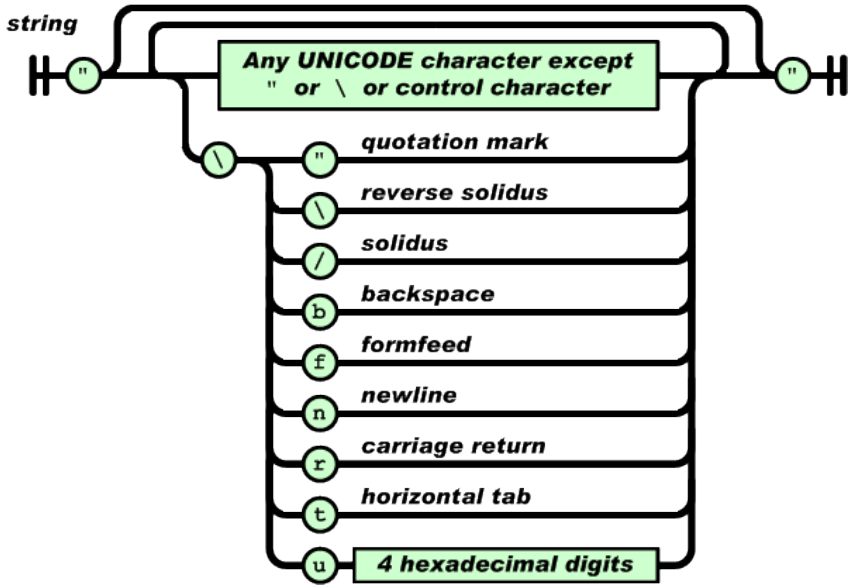
- JSON est un format de données textuelles permettant de représenter de l'information structurée.
- Il est dérivé de la notation des objets en JavaScript
- Crée par Douglas Crockford entre 2002 et 2005, décrit dans la RFC 4627
- Un document JSON comprend deux types d'éléments structurels :

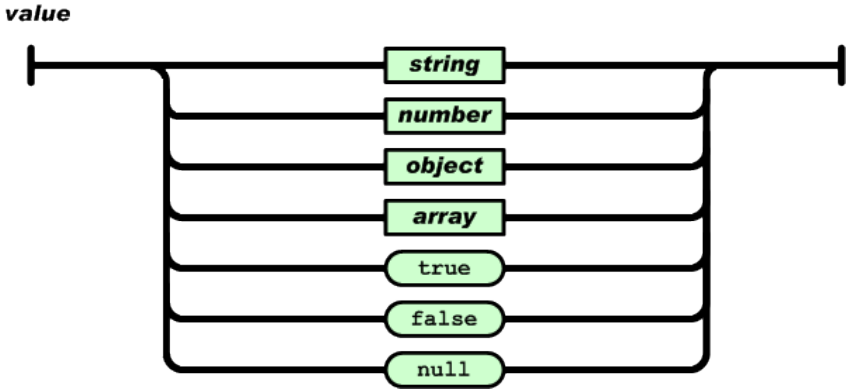


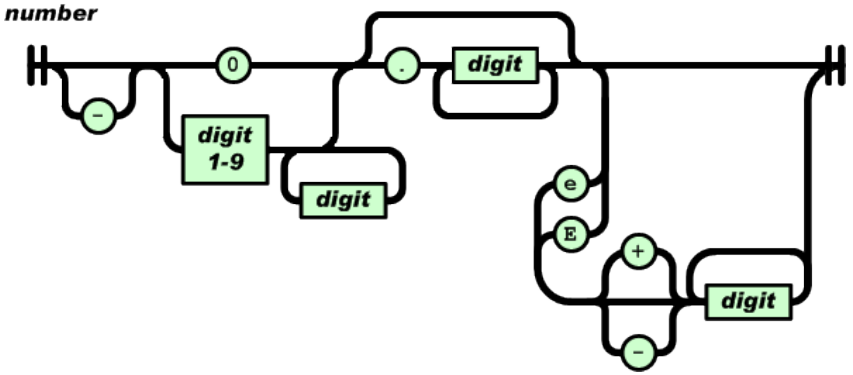


JSON (JavaScript Object Notation)

- Une valeur peut être :







JSON (JavaScript Object Notation)

- Les avantages du format JSON :
  - Indépendant des langages de programmation
  - Types de données simples à décrire
  - Bonne lisibilité de la syntaxe
  - Moins volumineux en termes de stockage que le XML et plus simple en termes de syntaxe
  - Pour JavaScript un document JSON représente un objet

```
:
{
  "marque": "Citroen",
  "modele": "C4",
  "caracteristiques": {
    "longueur": 3.80,
    "largeur": 1.90,
    "automatique": false
  },
  "couleur": ["blanc", "rouge", "vert", "bleu"]
}
```

BSON (Binary JSON)

- Représentation binaire des objets JSON
- Utilisé comme stockage de données et format de transfert de données par MongoDB
- Les types terminaux :
  - Octet (8 bits)
  - Entier signé (32 et 64 bits)
  - Réel (nombre à virgule flottante sur 64 bits) et Décimal haute précision (128 bits)
- Les autres types :
  - Chaîne de caractères (encodé en UTF-8)
  - Booléen
  - Date (UTC) et Timestamp
  - Expressions régulières
  - JavaScript
  - Null
  - Tableau
  - Sous document
  - Binaire

Types : identifiants & alias

Type	Num	Alias
Double	1	double
String	2	string
Object	3	object
Array	4	array
Binary data	5	binData
ObjectId	7	objectid
Boolean	8	bool
Date	9	date
Null	10	null
Regular Expression	11	regex
JavaScript	13	javascript
Javascript (w. scope)	15	javascriptWithScope
32-bit integer	16	int
Timestamp	17	timestamp
64-bit integer	18	long
Decimal128	19	decimal

ObjectId

- Les ObjectId sont des valeurs uniques constitués de 12 octets, dans l'ordre :
  - 4 octets : un timestamp (secondes depuis 1e 1er janvier 1970)
  - 5 octets : une valeur aléatoire
  - 3 octets : un compteur initialisé à une valeur aléatoire

- Bien que les valeurs ObjectId devraient augmenter au fil du temps, elles ne sont pas nécessairement monotones.
- En effet, ils :
  - Ne contiennent qu'une seconde de résolution temporelle, donc les valeurs ObjectId créées dans la même seconde n'ont pas un ordre garanti.
  - Sont générés par les clients, qui peuvent avoir des horloges système différentes.

Dates et Timestamps

- Les dates sont stockées en BSON sous la forme d'un entier signé sur 64 octets représentant le nombre de millisecondes depuis le 1er janvier 1970.
  - Soit une plage de dates représentable d'environ 290 millions d'années dans le passé et l'avenir.
  - Sont des dates UTC, donc sans information de fuseau horaire

- Les timestamps sont stockées en BSON sur 64 octets réparti dans cet ordre :
  - 32 octets : les secondes depuis le 1er janvier 1970
  - 32 octets : un nombre ordinal incrémentiel pour les opérations dans une seconde donnée

- Les timestamps sont plus prévus pour un usage interne de MongoDB (valeur unique, horodatage du journal d'opération, …).

- Dans la plupart des cas, nous utiliserons des dates dans nos applications.

MongoDB Extended JSON

- JSON ne peut représenter qu'un sous-ensemble des types pris en charge par BSON.

- Pour conserver les informations de type, MongoDB ajoute les extensions suivantes au format JSON:
  - **Mode strict :** représentations des types BSON conformes à la RFC JSON.
    L'analyseur JSON interne de MongoDB reconnaît les informations de type véhiculées par le format.
  - **Mode shell** : fonctions raccourcis gérées par le shell.

- Représentation du type « binary » :
  - `Strict : { "$binary": "<bindata>", "$type": "<t>" }`
  - Shell : `BinData(<t>, <bindata>)`

  - `bindata` : représentation base64 de la charge binaire
  - `t` : un octet représentant le type de binaire



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Installation de MongoDB

Chapitre 5



mongoDB.

Installation sous Windows

- Récupérer le programme d'installation sur le site de l'éditeur : https://www.mongodb.com/download-center
- A l'issue de l'installation, par défaut, MongoDb se situe dans le dossier « C:\Program Files\MongoDb\Server**version**\ »
- Au besoin, modifier le fichier de configuration « C:\Program Files\MongoDB\Server**version**\bin\mongod.cfg ». (préconfiguré pour un serveur isolé).
- Un service windows est généré et lancé automatiquement.
- N.B. : Les outils font l'objet d'une installation séparée.

Installation sous Linux

- Procédures d'installation de MongoDB sous linux disponibles à l'adresse : https://docs.mongodb.com/manual/administration/install-on-linux/

- Ces procédures varient en fonction de la distribution (RedHat, SUSE, Ubuntu, Debian, Amazon) mais suivent le même cheminement.

- Il est également possible d'installer MongoDB manuellement via une archive « tarball » (pour les systèmes d'exploitation supportés par MongoDB n'ayant pas de système de packages)

Installation sous Docker

Déclarer un nouveau repository pour docker :

```
sudo yum-config-manager --add-repo
https://download.docker.com/linux/centos/docker-ce.repo
```

Installer Docker :

```
sudo yum install docker-ce
```

Démarrer un service mongodb :

```
docker run --name some-mongo -d mongo:tag
```



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Les outils MongoDB

Chapitre 6

mongoDB

Les outils MongoDB

- A l'issu de l'installation de MongoDB, on dispose des exécutables suivants :
  - **mongod** : le moteur de base
  - **mongosh** : le shell
  - **mongos** : le contrôleur de sharding
  - Les outils d'import/export : **mongoimport**, **mongoexport**
  - Les outils de sauvegarde/restauration : **mongodump**, **mongorestore**
  - **bsondump** : conversion bson ➔ json
  - **mongofiles** : l'utilitaire GridFs
  - **mongostat** : aperçu du statut d'une instance **mongo** ou **mongos**
  - **mongotop** : état des temps de lecture / écriture actualisé chaque seconde
  - **mongoldap** : teste une connexion LDAP sur une instance (entreprise)
  - **mongokerberos** : valide la configuration Kerberos et la teste (entreprise)



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

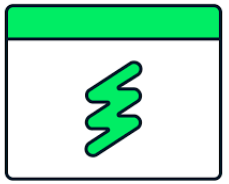
CONNEXION ET UTILISATION DE MONGODB

MODULE 2

mongo DB

Connexion et utilisation de MongoDB

- Utilisation du Shell Mongo
- Les bases, les collections, les documents et les espaces de noms
- Manipulation des objets et données : les opérations CRUD
- Ecrire des scripts pour le Shell Mongo





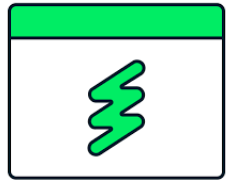


CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Utilisation du shell Mongo

Chapitre 1



Mongosh

- Le shell est accessible via la commande :

- L'adresse de la base doit être une chaine de connexion, par exemple :
  - "mongodb://mongodb0.example.com:27017/testdb?tls=true"

- Quelques options :
  - --port <port> : pour préciser un port différent de 27017
  - --host <hostname> : pour préciser un hôte différent de « localhost »
  - --file : pour exécuter un script JS

- Pour l'exécution de script, il faut savoir que mongosh embarque Node.js, on peut donc bénéficier de son écosystème.

Premières commandes d'administration

- Voici quelques commandes exécutables dans le shell mongo :
  - **show dbs** : pour afficher les bases de données existantes
  - **db** : retourne la base de données en cours
  - **use <db>** : pour utiliser une base de données (« test » par défaut sauf si précisé dans la ligne de commande)
  - **show collections** : pour afficher les collections dans la base en cours
  - **show users** : pour afficher les utilisateurs définis dans la base en cours
  - **show logs** : pour afficher les logs disponibles
  - **show log [name]** : pour afficher le contenu d'un log (« global » par défaut)
  - **exit** : pour sortir du shell

  - **help** : pour afficher un premier niveau d'aide
  - **db.help()** : aide des méthodes liées aux bases de données
  - **db.<collection>.help()** : aide des méthodes liées aux collections



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Les bases, les collections, les documents et les espaces de noms

Chapitre 2

Les bases de données

- Une base de données doit respecter quelques règles de nommage
  - Le nom ne peut être une chaîne vide ou le caractère « null »
  - Le nom ne peut pas contenir certains caractères :
    - `/\."$*<>:|?` et l'espace (sous windows)
    - `/\."$` et l'espace (sous unix/linux)
  - Le nom a une limite de 64 caractères
  - Le nom ne peut pas être celui d'une base « système » : admin, local, config
  - Les noms sont insensibles à la casse

Les bases de données

- Une base est créée automatiquement lors de la première insertion de données

```
mongosh <db-name>
> use <db-name>
```

- Pour supprimer la base en cours, utiliser la commande d'administration :

```
db.runCommand( { dropDatabase: 1 } )
```

  - Le shell propose également une fonction raccourcie : **db.dropDatabase()**
  - La commande ne supprime pas les utilisateurs définis dans la base. Utilisez **db.dropAllUsersFromDatabase()** avant !

Les collections

- Une collection est créée automatiquement lors de la première insertion d'un document

- On peut également la créer de manière explicite via la fonction **db.createCollection(name, [options])**
  Quelques options :
  - capped : true (pour définir une collection « cappé » c.a.d. limitée en taille : mode « FIFO »)
  - size : <nombre> (la taille maximum en octets d'une collection « cappé »)
  - max : <nombre> (le nombre maximum de documents d'une collection « cappé »)
  - validator : <document> (document qui précise des règles de validation)
  - validationLevel : <chaîne> (comment la base valide les documents)
    - « strict » : sur toutes les insertions et modifications, « moderate » : idem sauf les documents existants
  - validationAction : <chaîne> (ce que fait la base en cas d'erreur)
    - « error » : l'écriture échoue, « warn » : alerte dans les logs

Les collections

- Le nom de la collection ne peut :
  - être une chaine vide, contenir les caractères « `null` » et « `$` »
  - commencer par « `system.` »
    (sous-collections possibles, ex : `Blog.posts`, `Blog.authors`)
- Pour lister les collections de la base en cours :
  - `show collections;`
  - `db.getCollectionNames();`
- Pour supprimer une collection sur la base en cours :
  - `db.<nom_collection>.drop();`

Les collections « timeseries » (>=5.0)

- Une collection « timeseries » stocke des mesures au cours du temps de manière efficace. Ces mesures étant identifiées par des métadonnées qui change très rarement :

Sujet	Mesure	Métadonnées
Météo	Température	Numéro de capteur, localisation
Bourse	Valeur d'une action	Code de l'action, bourse
Fréquentation de site web	Nombre de vues	URL

- Stockées en interne de manière optimisé, ces collections optimisent les performances des requêtes, l'utilisation du cache correspondant et réduisent le volume de données (stocké, entrant, sortant).

Les collections « timeseries » (>=5.0)

- Une collection « timeseries » est créé de la manière suivante :

```
db.createCollection(name, {
  timeseries: [options],
  expireAfterSeconds: nombre
})
```

Quelques options :

- timeseries.timeField : le champ des documents qui contient la date de la mesure (obligatoire)
- timeseries.metaField : le champ des documents qui contient les métadonnées (optionnel)
- timeseries.granularity : la granularité des mesures : « seconds », « minutes » ou « hours » (optionnel, « seconds » par défaut : choisir la granularité s'approchant le plus de l'intervalle entre deux mesures)
- expireAfterSeconds : le nombre de secondes au bout duquel une mesure pourra être supprimée automatiquement (optionnel, pas d'expiration par défaut)

Les collections « clustered » (>=5.3)

- Une collection « clustered » stocke les documents dans l'ordre des valeurs de la clé / l'index de clustering.

- Les bénéfices sont les suivants :
  - Requêtes plus rapides filtrant sur des intervalles de valeurs de la clé (ou une égalité).
  - Economie d'espace de stockage et performances améliorées pour les insertions par lot.
  - Peut éliminer le besoin d'un index si le champ recherché est dans la clé de clustering.
  - Possibilité de TTL incluse, améliore les performances de suppression correspondantes.

- Il ne peut avoir qu'un seul index « clustered » par collection et il ne peut être crée qu'à la création de la collection.

Les collections « clustered » (>=5.3)

- Une collection « clustered » est créé via l'option « clusteredIndex » :

```
db.createCollection(name, {
  clusteredIndex: {
    key: { _id : 1 },     // obligatoire
    unique: true,         // obligatoire
    name: <string>        // optionnel : nom de l'index
  },
})
```

- Les incovénients sont les suivants :
  - Impossible de transformer une collection « non clustered » en « clustered » et inversement.
  - Si un autre index existe, il faut préciser que l'on souhaite utiliser l'index « clustered ».
  - Peut éliminer le besoin d'un index si le champ recherché est dans la clé de clustering.
  - Une collection « clustered » ne peut être « cappé ».

Les collections « clustered » (>=5.3)

- Les clé de custering :
  - doivent contenir des valeurs uniques.
  - doivent contenir des valeurs immuables.
  - devraient contenir des valeurs augmentant de manière séquentielle pour optimiser les insertions.
  - devraient aussi petites que possible pour éviter de pénaliser les performances et d'anéantir les avantages en termes de taille de stockage.

Les documents

- Un document peut être inséré dans une collection :
  - Via la méthode **db.<nom_collection>.insertOne(<document>);**
- MongoDB ajoute automatiquement une clé primaire unique pour chaque document dans le champ « _id » et crée un index sur celui-ci
- Les clés des documents ne doivent pas contenir « null », « . » ou commencer par « $ »
- Les clés doivent être uniques au sein d'un même document
- A noter que MongoDB est sensible à la casse
- Les documents sont exprimés au format JSON

Les espaces de noms

- Chaque collection et chaque index compte comme un « espace de nom »
- Plus concrètement la concaténation du nom de la base et du nom de la collection / de l'index
  - Ex: <database>.<collection>
- La longueur d'un espace de nom est au maximum de 255 octets.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Manipulation des objets et données : les opérations CRUD

- Chapitre 3

Les instructions CRUD à travers le shell – Create

- La création de contenu dans les collections se fait via les fonctions :

```
db.<collection>.insertMany(
     [<document>,…], {
     writeConcern: <document>,
     ordered: <boolean>
})
db.<collection>.insertOne(
     <document>, {
     writeConcern: <document>
})
```

- **writeConcern** : document représentant la stratégie de prise en compte de l'écriture
- **ordered** :
  - Si vrai, insère les documents du tableau dans l'ordre et interromps les insertions en cas d'erreur.
  - Si faux, insère les documents du tableau dans n'importe quel ordre et continue les insertions en cas d'erreur.

- Quelques exemples :

```
db.produits.insertOne({name:"lecteur DVD", prix: 199.99})
db.produits.insertMany([{name:"lecteur DVD", prix: 199.99}, ... ])
```

Les instructions CRUD à travers le shell – Read

- On peut lire des documents depuis une collection avec la méthode « find » :

```
db.<collection>.find
```

- Si on ne fournit pas de paramètres, l'ensemble des documents de la collection est retourné.

- On peut préciser les documents que l'on souhaite en précisant un filtre sous la forme d'un document JSON exprimant les conditions que les documents doivent remplir pour être retournés.

- Quelques exemples :

```
db.produits.find({ type: "canapé" });
db.produits.find({ type: "canapé", matiere: "cuir" });
db.produits.find({ type: "canapé", matiere: "cuir", couleur: "noir" });
```

Les instructions CRUD à travers le shell – Read

- Il est possible d'effectuer des recherches plus élaborées en utilisant des opérateurs à la place des valeurs.

- Opérateurs de comparaison :
  - `{$gt: <value>}` : supérieur à
  - `{$gte: <value>}` : supérieur ou égal à
  - `{$lt: <value>}` : inférieur à
  - `{$lte: <value>}` : inférieur ou égal à
  - `{$ne: <value>}` : différent de
  - `{$in: [<value1>, <value2>, <value3>]}` : compris dans
  - `{$nin : [<value1>, <value2>, <value3>]}` : non compris dans

- Quelques exemples :

```
db.produits.find({ type: "canapé", prix: {$lt: 600} });
db.produits.find({ couleur: {$in: ["noir", "blanc"]} });
```

Les instructions CRUD à travers le shell – Read

- Il est possible d'effectuer des recherches plus élaborées en utilisant des opérateurs à la place des valeurs.

- Opérateurs logiques :

```
{$or: [{ <expression1> }, { <expression2> }, ..., { <expressionN> }] } : OU
{$and: [{ <expression1> }, { <expression2> }, ..., { <expressionN> }] } : ET
{$not: { <expression> } } : NON
{$nor: [{ <expression1> }, { <expression2> }, ..., { <expressionN> }] } : NON OU (tout doit être faux)
```

- Quelques exemples :

```
db.produits.find( {$and: [
  {prix: {$gte: 600}},
  {prix: {$lt: 900}}
]} );
```

Les instructions CRUD à travers le shell – Read

- Il est possible d'effectuer des recherches plus élaborées en utilisant des opérateurs à la place des valeurs.

- Opérateurs sur les éléments :
  - `{$exists: <booléen> }` : le champ doit exister ou ne pas exister
  - `{$type: <nombre identifiant> | <chaine alias> }` : le champ doit être du type https://docs.mongodb.com/manual/reference/operator/query/type/#op._S_type

- Opérateurs sur les tableaux :
  - `{$all: [ <value1> , <value2> ... ] }` : le tableau doit contenir toutes les valeurs
  - `{$elemMatch: { <query1>, <query2>, ... } }` : au moins un élément du tableau doit répondre aux critères
  - `{$size: <nombre> }` : doit contenir x éléments
- Un exemple :

```
db.scores.find({ results: { $elemMatch: { $gte: 14, $lt: 16 } } })
```

Les instructions CRUD à travers le shell – Read

- Au cas où les opérateurs de recherche ne suffisent pas pour exprimer vos filtres, il existe également l'opérateur `$expr` (>= 3.6)

- Celui-ci se base sur les opérateurs du pipeline d'agrégation.
  - Ex: comparer deux champs d'un même document.

```
{ "_id" : 1, "categorie" : "alimentaires", "budget": 400, "depense": 450 }
{ "_id" : 2, "categorie" : "boissons", "budget": 100, "depense": 150 }
{ "_id" : 3, "categorie" : "vetements", "budget": 100, "depense": 50 }
{ "_id" : 4, "categorie" : "autres", "budget": 500, "depense": 300 }
{ "_id" : 5, "categorie" : "voyages", "budget": 200, "depense": 650 }

db.budgetMensuel.find( { $expr: { $gt: [ "$depense" , "$budget" ] } } )

{ "_id" : 1, "categorie" : "alimentaire", "budget" : 400, "depense" : 450 }
{ "_id" : 2, "categorie" : "boisson", "budget" : 100, "depense" : 150 }
{ "_id" : 5, "categorie" : "voyages", "budget" : 200, "depense" : 650 }
```

Les instructions CRUD à travers le shell – Read

- Il est possible de restreindre les données renvoyées par document via une projection :

```
db.<collection>.find(<query>, { field1: <value>, field2: <value> ... });
```

- On précise les champs à inclure ou à exclure du résultat
- Pour préciser cela, on donne comme valeur au champ :
  - 1 ou vrai pour inclure le champ
  - 0 ou faux pour exclure le champ
- On ne peut pas préciser à la fois des champs à inclure et des champs à exclure
- Le champ « _id » est inclus par défaut, sauf si on l'exclut explicitement

Les instructions CRUD à travers le shell – Curseur et boucle

- La fonction « db.collection.find » renvoie un curseur sur lequel il faut itérer pour accéder aux documents.
- Si le curseur n'est pas affecté à une variable, celui-ci est itéré automatiquement par le shell pour afficher les 20 premiers résultats.

```
var monCurseur = db.stocks.find();
while(monCurseur.hasNext()) {
  printjson(monCurseur.next());
}
db.contacts.find().forEach(function(doc){print(doc.nom);});
```

- On peut également parcourir le résultat comme un tableau :

```
var monCurseur = db.stocks.find();
var docArray = monCurseur.toArray();
print(docArr[0].nom);
```

Les instructions CRUD à travers le shell – Tri

- La fonction « cursor.sort » permet de trier les résultats et doit être appliqué avant de commencer à récupérer les documents.
  - Le tri peut se faire sur un ou plusieurs champs :

```
monCurseur.sort({ age: 1 });
monCurseur.sort({ age: 1, note: -1 });
```

- A moins que MongoDB puisse s'appuyer sur un index pour faire le tri, il refuse d'utiliser plus de 100Mo de mémoire et renvoie une erreur dans ce cas.

- Pour éviter cela, il faut créer un index ou utiliser la fonction « limit » pour restreindre le jeu de résultat : `monCurseur.sort({ age: 1 }).limit(10);`
  - On peut également autoriser l'écriture de fichier sur le disque :

```
monCurseur.sort({ age: 1 }).allowDiskUse();
```

Les instructions CRUD à travers le shell – Collation

- La fonction « cursor.collation » permet de définir la logique de classement en fonction d'une langue.
  - L'information obligatoire à fournir est la langue demandée :

```
monCurseur.collation({ locale: "fr" });
```

- D'autres paramètres sont renseignables, mais la locale les prédéfinit aux valeurs adéquates pour la langue choisie.

- Plus d'infos : https://docs.mongodb.com/manual/reference/collation/

Les instructions CRUD à travers le shell – findOne

- La fonction « findOne » permet de récupérer un document d'une collection :

```
db.<collection>.findOne(<query>, <projection>)
```

- Les deux arguments sont optionnels :
  - `query` : le filtre de recherche
  - `projection` : la projection

- Comme la fonction retourne un document, il n'est pas possible de trier : le document retourné est le 1er trouvé par le filtre dans l'ordre d'insertion en base.

Les instructions CRUD à travers le shell – Update

- La modification de contenu dans les collections se fait via les fonctions :

```
db.<collection>.updateOne(<query>, <update>, {
     upsert: <booleen>,
     writeConcern: <document>
})
db.<collection>.updateMany(<query>, <update>, {
     upsert: <booleen>,
     writeConcern: <document>
})
db.<collection>.replaceOne(<query>, <replacement>, {
     upsert: <booleen>,
     writeConcern: <document>
})
```

  - **query** : un document représentant les critères de sélection pour déterminer les documents à modifier
  - **update** : un document représentant les modifications à apporter
  - **upsert** : si vrai, insère un nouveau document si les critères de sélection ne renvoient aucun document à modifier (défaut : faux)
  - **writeConcern** : document représentant la stratégie de prise en compte de l'écriture

Les instructions CRUD à travers le shell – Update

- Il est possible d'effectuer des modifications élaborées en utilisant des opérateurs :
  - `$set: { <field1>: <value1>, ... }` : modifie les champs spécifiés
  - `$inc: { <field1>: <amount1>, <field2>: <amount2>, ... }` : incrémente des champs d'une certaine valeur
  - `$mul: { field: <number> }` : multiplie un champ par une valeur
  - `$rename: { <field1>: <newName1>, <field2>: <newName2>, ... }` : renomme des champs
  - `$unset: { <field1>: "", ... }` : supprime des champs
  - `$currentDate: { <field1>: <typeSpecification1>, ... }` : insère la date courante
  - …

Les instructions CRUD à travers le shell – Update

- Quelques exemples :

```
db.produits.replaceOne({_id:"123456789"},{marque:"Sony", modele:"Xperia"});

db.produits.updateMany({marque:"Sony Ericsson"},{$set : {marque: "Sony"} });

db.users.updateOne({ _id: 1 },
   { $currentDate: {
        lastModified: { $type: "date" }, // => Date UTC
        "cancellation.date": { $type: "timestamp" } // => Timestamp
     },
     $set: {
        status: "D",
        "cancellation.reason": "user request"
     }
   });
```

Les instructions CRUD à travers le shell – Update

- On peut fournir un pipeline d'agrégation en 2ème paramètre, ex:

```
db.collection.updateMany(
   <query>,
   [
      { $set: { status: "Modified", comments: [ "$misc1", "$misc2" ] } },
      { $unset: [ "misc1", "misc2" ] }
   ]
   ...
)
```

- Seulement certaines étapes sont utilisables : `$addFields/$set, $project/$unset, $replaceRoot/$replaceWith.`

Les instructions CRUD à travers le shell – findOneAndReplace

- La fonction « findOneAndReplace » permet de chercher et remplacer un document :

```
db.<collection>.findOneAndReplace(<filter>, <replacement>, {
  sort: <document>,
  projection: <document>,
  upsert: <boolean>,
  returnNewDocument: <boolean>,
  collation: <document>
})
```

- **filter** : le filtre de recherche
- **remplacement** : le document de remplacement
- **sort** : l'ordre de tri
- **projection** : projection pour le document de retour
- **upsert** : si vrai, insère un nouveau document si les critères de sélection ne renvoient aucun document à modifier (défaut : faux)
- **returnNewDocument** : si vrai, retourne le document de remplacement plutôt que le document remplacé (défaut : faux)
- **collation**: pour paramétrer le classement en fonction d'une langue

Les instructions CRUD à travers le shell – findOneAndUpdate

- La fonction « findOneAndupdate » permet de chercher et modifier un document :

```
db.<collection>.findOneAndUpdate(<filter>, <update>, {
  sort: <document>,
  projection: <document>,
  upsert: <boolean>,
  returnNewDocument: <boolean>,
  collation: <document>,
  arrayFilters: [ <filterdocument1>, ... ]
})
```

- **filter** : le filtre de recherche
- **update** : le document de modification (ou un pipeline d'aggregation : >= 4.2)
- **sort** : l'ordre de tri
- **projection** : projection pour le document de retour
- **upsert** : si vrai, insère un nouveau document si les critères de sélection ne renvoient aucun document à modifier (défaut : faux)
- **returnNewDocument** : si vrai, retourne le document modifié plutôt que le document d'origine (défaut : faux)
- **collation**: pour paramétrer le classement en fonction d'une langue

Les instructions CRUD à travers le shell – Delete

- La suppression de documents se fait via la fonction suivante :

```
db.<collection>.deleteOne(<query>, {
            writeConcern: <document>
});
db.<collection>.deleteMany(<query>, {
            writeConcern: <document>
});
```

- query : document représentant les critères de sélection pour la suppression
- writeConcern : document représentant la stratégie de prise en compte de l'écriture

- Quelques exemples :

```
db.stocks.deleteMany({});
db.stocks.deleteMany({type: "périssable"});
db.stocks.deleteOne({code: "123456789"});
```

Les instructions CRUD à travers le shell – findOneAndDelete

- La fonction « findOneAndDelete » permet de chercher et supprimer un document :

```
db.<collection>.findOneAndDelete(<filter>, {
  sort: <document>,
  projection: <document>,
  collation: <document>
})
```

- **filter** : le filtre de recherche
- **sort** : l'ordre de tri
- **projection** : projection pour le document de retour
- **collation**: pour paramétrer le classement en fonction d'une langue



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Ecrire des scripts pour le shell Mongo

Chapitre 4

Ecrire des scripts pour le shell Mongo

- Il est possible d'exécuter des scripts JavaScript via le client « mongo », ex :

```
mongosh --file myScript.js
```

Il est également possible d'exécuter un fichier JavaScript depuis le shell Mongo, ex :

```
> load("myScript.js")
```

- A l'intérieur de votre fichier JavaScript, vous pouvez instancier une connexion vers un serveur MongoDB via le constructeur « Mongo » :

```
conn = new Mongo()
conn = new Mongo(<host>)
conn = new Mongo(<host:port>)
```

- Sélectionnez ensuite la base de données :

```
db = conn.getDB("myDatabase");
```

- Ou plus simplement via la fonction « connect » :

```
db = connect("localhost:27020/myDatabase");
```

Ecrire des scripts pour le shell Mongo

- Il n'est pas possible d'utiliser les aides du shell, seulement du Javascript valide :

```
show dbs ➔ db.adminCommand({listDatabases:1})
use <db> ➔ db = db.getSiblingDB('<db>')
show collections ➔ db.getCollectionNames()
show logs ➔ db.adminCommand({ 'getLog' : '*' })
show log <logname> ➔ db.adminCommand({ 'getLog' : '<logname>' })
…
```

- Pour parcourir des résultats, utilisez les curseurs.

- Utilisez les fonctions « print » et « printjson » pour afficher dans la console.

```
conn = new Mongo("localhost.localdomain");
db = conn.getDB("local");
print("switch to local");
cursor = db.startup_log.find();
while ( cursor.hasNext() ) { printjson(cursor.next()); }
```

.mongoshrc.js

- Au lancement, le shell cherche un fichier « .mongoshrc.js » dans le « user home ».
- Exécuté si présent avant l'affichage du prompt.
- Si un fichier.js est dans la ligne de commande, il est exécuté puis le « .mongoshrc.js » à son tour (--rc pour éviter cela)
- Ce fichier peut servir à personnalisé le prompt par exemple :

```
host = db.serverStatus().host;
prompt = function() { return db.getName() + "@" + host + "$ "; }
```

- Ou encore à configurer le nombre de documents itérés automatiquement :

```
DBQuery.shellBatchSize = 10;
```



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Le driver C#

MODULE 3

Gestion des drivers

- Composantes et compatibilité de versions
- Connexion aux serveurs MongoDB et choix d'une base de données
- Les instructions CRUD via le driver en utilisant LINQ
- Intégration de MongoDB dans l'ORM Entity Framework, utilisation et limitations



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Les drivers disponibles pour MongoDB.

Chapitre 1

Les drivers disponibles pour MongoDB

- L'éditeur de MongoDB fournit des drivers pour les langages :
  - C
  - C++
  - C#
  - Go
  - Java
  - Node.Js
  - PHP
  - Python
  - Ruby
  - Rust
  - Scala
  - Swift
- Des drivers communautaires sont également disponibles pour d'autres langages.

Présentation du driver C# – MongoDB.Driver

- Le driver C# est la bibliothèque officielle développée par MongoDB Inc.
- API typée pour le framework .NET

Fonctionnalités clés du driver C#:

- Connexion simplifiée à la base
- Accès direct aux bases et collections
- Exécution d'opérations CRUD asynchrones
- Support complet de LINQ
- Gestion native des transactions, index et agrégations

Compatibilité

- Compatibilité avec MongoDB

```
MongoDB 5.0 → C# Driver ≥ 2.14
MongoDB 6.0 → C# Driver ≥ 2.18
MongoDB 6.1 → C# Driver ≥ 2.19
MongoDB 7.0 → C# Driver ≥ 2.22
MongoDB 8.0 → C# Driver ≥ 2.25
```

- Compatibilité avec .NET

```
Driver ≤ 2.14 → .NET Framework 4.5 / .NET Core 3.1
Driver ≥ 2.15 → .NET 5, 6, 7
Driver ≥ 2.22 → .NET 8 compatible
```

Installation du driver MongoDB pour .NET

- Installe le package officiel avec la commande :

```
Install-Package MongoDB.Driver
```

- le Gestionnaire de packages NuGet dans l'interface graphique

- Compatibilité



- Compatible avec .NET 6+, .NET Core, et .NET Framework 4.6+.
- Fonctionne sur Windows, Linux et macOS.
- Fournit des API synchrones et asynchrones (async/await) pour de hautes performances.

Connexion avec MongoClient

- Pour interagir avec MongoDB dans .NET, on utilise la classe MongoClient fournie par le package MongoDB.Driver.

```
using MongoDB.Driver;
var client = new MongoClient("mongodb://localhost:27017");
```

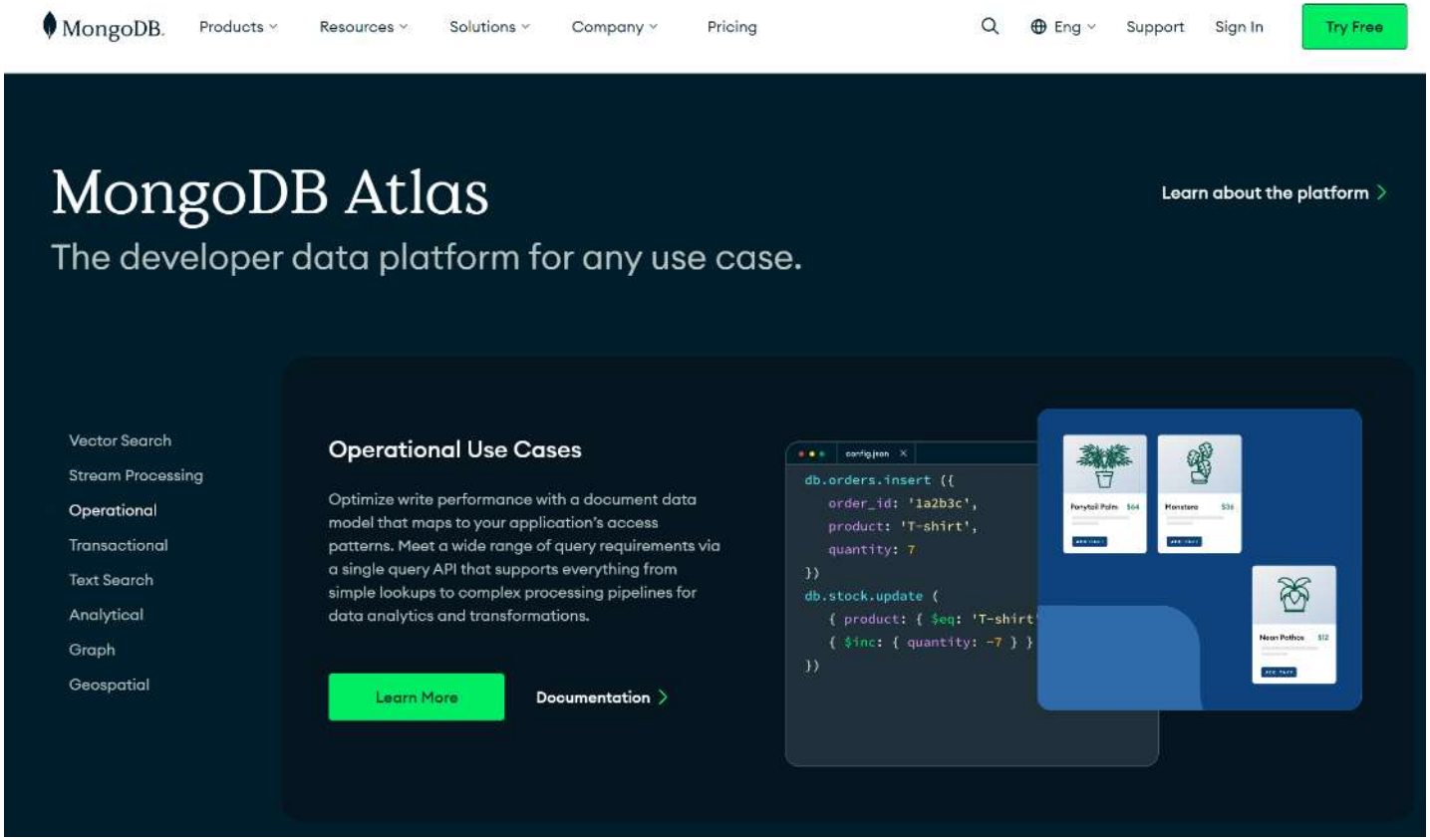
- URI de connexion

```
mongodb://[utilisateur:motdepasse@]hôte[:port]/[base]?options
```

- Accès à la base et à une collection

```
var database = client.GetDatabase("MaBase");
var collection = database.GetCollection<BsonDocument>("Utilisateurs");
```

MongoDB Atlas



Sélection d’une base et d’une collection

- Accéder à une base de données GetDatabase() :

```
var client = new MongoClient("mongodb://localhost:27017");
var database = client.GetDatabase("MaBase");
```

- Accéder à une collection GetCollection<T>()

```
var collection = database.GetCollection<BsonDocument>("Utilisateurs");
```

- Créer une collection

```
await database.CreateCollectionAsync("Utilisateurs");
```



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Les instructions CRUD en C# : Create, Read, Update, Delete

Chapitre 2

Les instructions CRUD en C# – Create

- Pour insérer un seul document :

```
collection.InsertOne(document);
```

- En async

```
await collection.InsertOneAsync(document);
```

- Pour insérer plusieurs documents en une seule opération

```
collection.InsertMany(documents);

await collection.InsertManyAsync(documents);
```

Les instructions CRUD en C# – Create

- Exemple

```
var users = new List<BsonDocument>
{
 new BsonDocument { { "name", "John" }, { "age", 28 } },
 new BsonDocument { { "name", "Albert" }, { "age", 32 } }
};
await collection.InsertManyAsync(users);
```

Notez que si la clé primaire « _id » n'est pas précisée dans le document, celle-ci sera insérée automatiquement

Les instructions CRUD en C# – Read

- Pour récupérer tous les documents :

```
var results = collection.Find(new BsonDocument()).ToList();
```

- Pour récupérer certains documents seulement en précisant un filtre comme document :

```
var filter = new BsonDocument
{
    { "stars", new BsonDocument { { "$gte", 2 }, { "$lt", 5 } } },
    { "categories", "Bakery" }
};
var results = collection.Find(filter).ToList();
```

  - En utilisant le Builders :

```
var filter = And(Gte("stars", 2), Lt("stars", 5), Eq("categories", "Bakery"));
var results = collection.Find(filter).ToList();
```

- Les fonctions `Eq, Gte, Lt, And`, etc. proviennent du namespace :

```
MongoDB.Driver.Builders.Filter
```

Les instructions CRUD en C# – Read

- **Projection avec un document BSON**

```
var projection = new BsonDocument
{
    { "name", 1 },
    { "stars", 1 },
    { "categories", 1 },
    { "_id", 0 }
};


var results = collection
     .Find(new BsonDocument())
   .Project(projection)
   .ToList();
```

Les instructions CRUD en C# – Read

- **Projection avec Builders**

```
var projection = Include("name")
    .Include("stars")
    .Include("categories")
    .Exclude("_id");

var results = collection
  .Find(new BsonDocument())
    .Project(projection)
    .ToList();

Pour utiliser ces fonctions « Builders», il faut les importer statiquement au
préalable
import static com.mongodb.client.model.Projections.*;
```

Les instructions CRUD en C#– Read

- Trier via un document BSON

```
var sort = new BsonDocument
{
    { "name", 1 },
    { "stars", -1 },
    { "categories", 1 }
};

var results = collection
    .Find(new BsonDocument())
    .Sort(sort)
    .ToList();
```

- La méthode Sort() permet de définir l'ordre de tri des résultats

- Valeur 1 => croissant, -1 => décroissant.

Les instructions CRUD en C#– Read

- Trier avec le Builder Sorts

```
var sort = Builders<BsonDocument>.Sort
    .Ascending("name")
    .Ascending("categories")
    .Descending("stars");

var results = collection
    .Find(new BsonDocument())
    .Sort(sort)
    .ToList();
```

Les instructions CRUD en C#– Read

- Parcours des résultats via une boucle « for each » :

```
foreach (var document in results)
    {
     Console.WriteLine(document.ToJson());

    }
```

  - Les valeurs sont extraites grâce à l'indexeur du BsonDocument :

```
document["active"].AsBoolean;
document["birthDate"].ToUniversalTime();
document["length"].AsDouble;
document["length"].AsInt32;
document["subDoc"].AsBsonDocument;
document["table"].AsBsonArray;
```

Requêtes LINQ

- Utiliser LINQ avec MongoDB
- permet d'interroger les collections directement via LINQ,
- une syntaxe plus lisible et typée.
- Avantages LINQ:
- Syntaxe fortement typée et intuitive.
- Traduit automatiquement les expressions en requêtes MongoDB.
- Compatible avec `Where, Select, OrderBy, GroupBy, Count`, etc.

Requêtes LINQ – Select()/Where()

- La méthode Select()

```
var projection = collection
    .AsQueryable()
    .Where(doc => doc["stars"].AsInt32 >= 3)
    .Select(doc => new { Name = doc["name"].AsString, Stars =
doc["stars"].AsInt32 })
.ToList();
```

- La méthode Where()

```
var results = collection
    .AsQueryable()
    .Where(doc => doc["stars"].AsInt32 >= 3 && doc["categories"].AsString == "Bakery")
    .ToList();
```

Les instructions CRUD en C# – Update

- Pour remplacer entièrement un document (ReplaceOne) :

```
var filter = new BsonDocument { { "name", "Alice" } };
var replacement = new BsonDocument
{
    { "name", "Alice" },
    { "age", 30 },
    { "city", "Paris" }
};
```

- collection.ReplaceOne(filter, replacement);

Les instructions CRUD en C# – Update

- En utilisant le Builders :

```
var filter = Builders<Person>.Filter.Eq(p => p.Name, "Alice");
var update = Builders<Person>.Update
    .Set(p => p.Age, 30)
    .Set(p => p.City, "Paris");

collection.UpdateOne(filter, update);
```

- Les fonctions `Eq, Set, Inc, Unset,` etc. proviennent du namespace :

```
MongoDB.Driver.Builder
```

Les instructions CRUD en C# – Update

- Avec l'option Upsert :

```
var filter = Builders<Person>.Filter.Eq(p => p.Email,
"bob@acme.com");
var update = Builders<Person>.Update
    .Set(p => p.Name, "Bob")
    .SetOnInsert(p => p.CreatedAt, DateTime.UtcNow);

var options = new UpdateOptions { IsUpsert = true };

collection.UpdateOne(filter, update, options);
```

Les instructions CRUD en C# – DELETE

- Pour supprimer un seul document (DeleteOne) :

```
var filter = new BsonDocument { { "name", "Alice" } };

collection.DeleteOne(filter);
Pour supprimer plusieurs documents (DeleteMany) :
var filter = new BsonDocument { { "status", "inactive" } };

collection.DeleteMany(filter)
```

Les instructions CRUD en C# – DELETE

- En utilisant le Builders :

```
var filter = Builders<Person>.Filter.Eq(p => p.City, "Paris");

 collection.DeleteMany(filter);
```

- Les fonctions `Eq, Gt, Lt, And, Or,` etc. proviennent du namespace : MongoDB.Driver.Builders.Filter

Les instructions CRUD en C# – DELETE

- Exemple combiné :

```
var filter = Builders<Person>.Filter.And(
    Builders<Person>.Filter.Eq(p => p.City, "Paris"),
    Builders<Person>.Filter.Lt(p => p.Age, 18)
);

collection.DeleteMany(filter);
```

- Vérifier le résultat de suppression :

```
var result = collection.DeleteMany(filter);

Console.WriteLine($"{result.DeletedCount} document(s) supprimé(s)");
```

Les instructions CRUD en C# – DELETE

Bonnes pratiques :

- Utiliser `DeleteOne` pour cibler précisément un document.
- Utiliser `DeleteMany` pour des nettoyages ou archivages en masse.

- Sauvegarder ou archiver avant suppression définitive (pas de rollback).

- Toujours tester le filtre avant la suppression
  `(Find(filter)` → pour vérifier les documents concernés)

Entity Framework & MongoDB

- Utiliser une approche ORM (Object-Relational Mapping) avec MongoDB pour faciliter les accès aux données via des entités C# typées.
- Le driver officiel MongoDB.Driver
- Des ORM alternatifs compatibles NoSQL, ex. :

```
MongoFramework
MongoDB.EntityFrameworkCore
NoEntityFramework.MongoDB
```

Entity Framework & MongoDB

- Example

```
var users = dbContext.Users.Where(u => u.IsActive).ToList();
```

- Requêtes typées via LINQ
- Sérialisation automatique C# ⇔ BSON
- Pas de migration SQL requise
- Compatible avec .NET 6+



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

MODÉLISATION ET INDEXATION

MODULE 4

Modélisation et indexation

- Le schéma design : réflexion sur la modélisation à utiliser
- L'indexation sous MongoDB
- L'optimisation des requêtes et le profiling
- L'analyse des plans d'exécution
- Fonctionnement de MongoDB : transaction, isolation, verrou…
- Gestion des performances sous MongoDB



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Le schéma design : réflexion sur la modélisation à utiliser

Chapitre 1

Le schéma de données

- MongoDB n'impose pas de schéma fixe pour les données au sein d'une collection.
- Chaque document peut être inséré dans une collection avec une structure différente.
- En général, les documents d'une collection partagent une structure similaire : la flexibilité facilite les évolutions fonctionnelles.
- Le principal défi de la modélisation des données consiste à équilibrer les besoins de l'application, les caractéristiques de performance du moteur de base de données et les modèles de récupération de données.
- Lors de la conception de modèles de données, il faut toujours tenir compte de l'utilisation des données par les applications (requêtes, mises à jour et traitement des données) ainsi que de la structure inhérente aux données elles-mêmes.

Les différents scénarios de schémas

- La décision clé dans la conception de modèles de données pour les applications MongoDB tourne autour de la structure des documents et comment l'application représente les relations entre les données.

- Il existe deux outils qui permettent aux applications de représenter ces relations:
  - Les documents intégrés
  - Les références

Les documents intégrés

- Les documents intégrés saisissent les relations entre les données en stockant les données associées dans une structure de document unique.
- Les documents MongoDB permettent d'incorporer des structures de documents dans un champ ou un tableau dans un document.
- Ces modèles de données « dénormalisées » permettent aux applications de récupérer et de manipuler des données associées dans une seule opération de base de données.



Les documents intégrés

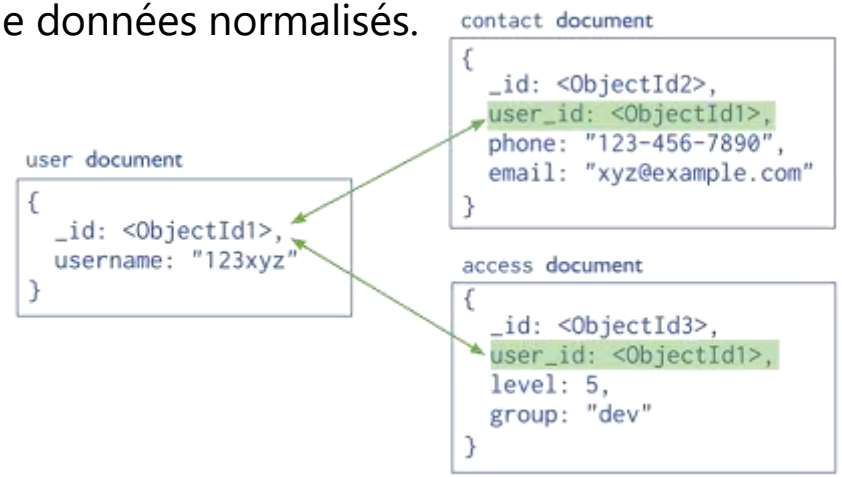
- En général, on utilise des documents intégrés lorsque :
  - Il y a une relation « A UN » (One-to-One) entre les entités.
  - Il y a une relation « A DES » (One-to-Many) entre les entités et que les entités enfants sont très dépendantes de l'entité parente.
    (c.a.d. lorsque ces entités enfants sont toujours présentées avec l'entité parente)

Les références

- Les références stockent les relations entre les données en incluant des liens ou des références d'un document à un autre.

- Les applications doivent résoudre ces références pour accéder aux données associées. (MongoDB ne supportent pas les jointures ➔ au moins deux requêtes doivent être effectuées)

- Il s'agit de modèles de données normalisés.



Les références

- En général, on utilise des références lorsque :
  - L'intégration engendrerait trop de duplications de données par rapport au bénéfice en terme de performance de requête.
  - Il y a une relation « A DES » bidirectionnelle (Many-to-Many) entre les entités.
  - On a une grande hiérarchie d'entités

La validation de document

- La version 3.2 de MongoDB a ajouté la possibilité de valider des documents lors de leurs créations et de leurs mises à jour.

- Cette validation se fait par collection et se définit lors de la création de la collection (ou sur une collection existante via la commande d'administration « `collMod` »)

- Les règles de validation sont exprimables avec l'ensemble des opérateurs de recherche sauf `$near, $nearsphere, $text` et `$where`

- Le paramètre « `validationLevel` » permet de préciser si la validation s'effectue :
  - sur toutes les créations et mises à jour (= `strict`)
  - uniquement sur les documents qui sont valides avant la modification (= `moderate`)

- Le paramètre « `validationAction` » permet de préciser si une violation des règles :
  - entraîne une erreur et rejette le document fautif (= `error`)
  - écrit un avertissement dans les logs mais autorise la création/mise à jour (= `warn`)

La validation de document

- Exemple via le shell :

```
db.createCollection( "contacts",
   {
      validator: { $or:
         [
            { phone: { $type: "string" } },
            { email: { $regex: /@mongodb\.com$/ } },
            { status: { $in: [ "Unknown", "Incomplete" ] } }
         ]
      },
      validationAction: "warn",
      validationLevel: "moderate"
   }
)
```

La validation de document

- La version 3.6 de MongoDB permet de mettre en place de schémas JSON.

- Via l'opérateur « `$jsonSchema` » il est possible d'exprimer d'autres règles par champs :
  - bsonType : un alias de type ou un tableau d'alias de type imposé
  - enum : un tableau de valeurs possibles
  - allOf : un tableau de schémas qui doivent être **tous** respectés
  - anyOf : un tableau de schémas dont **au moins un** doit être respecté
  - oneOf : un tableau de schémas dont **un seul doit** être respecté
  - not : un tableau de schémas dont **aucun** ne doit être respecté

- Plus spécifiquement pour les nombres :
  - multipleOf : doit être un multiple de
  - maximum : ne doit pas dépasser un maximum
  - minimum : ne doit pas être inférieur à un minimum
  - exclusiveMaximum : si le maximum doit être exclus (> … >=)
  - exclusiveMinimum : si le minimum doit être exclus (> … >=)

La validation de document

- Pour les chaines de caractères :
  - maxLength : nombre de caractères maximum
  - minLength : nombre de caractères minimum
  - pattern : une expression régulière à respecter
- Pour les objects :
  - properties : objet contenant les champs de l'objet avec pour chacun d'eux comme valeur un schéma JSON
  - required : tableau des noms des champs obligatoires
  - additionalProperties : si l'objet accepte des champs supplémentaires
  - maxProperties : nombre maximum de champs
  - minProperties : nombre minimum de champs
  - patternProperties : idem « properties » sauf que les noms des champs sont exprimés sous forme d'expressions régulières
- Pour les tableaux :
  - items : schéma JSON ou tableau de schéma JSON que les éléments doivent respecter
  - maxItems : nombre maximum d'éléments
  - minItems : nombre minimum d'éléments
  - uniqueItems : si les éléments doivent être uniques

La validation de document

- Exemple :

```
db.createCollection("diplomes", {
   validator: {
      $jsonSchema: {
         bsonType: "object",
         required: [ "nom", "annee", "filiere" ],
         properties: {
            nom: { bsonType: "string"},
            genre: { bsonType: "string"},
            annee: {
               bsonType: "int",
               minimum: 1958,
               maximum: 2017
        },
            filiere: { enum: [ "Maths", "Lettres", "Informatique", "Histoire", null ]}
         },
         additionalProperties: false
}}})
```



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Validation avec C#

Chapitre 2

Validation des documents avec C#

- La validation s'effectue avant l'insertion dans MongoDB.
  On utilise les attributs de validation (`DataAnnotations`) sur les classes C# (`POCO – Plain Old CLR Object`).

- Attributs courants :
- `[Required]` → champ obligatoire
- `[Range(min, max)]` → plage de valeurs autorisée
- `[EmailAddress]` → format d'e-mail valide
- `[StringLength(max)]` → longueur maximale de chaî

Validation des documents avec C#

- Example

```
public class Client
{
    [Required]
    public string Name { get; set; }

    [EmailAddress]
    public string Email { get; set; }

    [Range(18, 65)]
    public int Age { get; set; }
}
```

Validation des documents avec C# (Validator)

- Validation avant insertion :

```
var client = new Client { Name = "", Email = "test@", Age = 10 };
var context = new ValidationContext(client);
var results = new List<ValidationResult>();

bool isValid = Validator.TryValidateObject(client, context, results, true);

if (!isValid)
{
    foreach (var error in results)
        Console.WriteLine($"Erreur : {error.ErrorMessage}");
}
```

Validation manuelle et logique métier

- Permet d'ajouter des règles de validation personnalisées impossibles à exprimer avec les simples attributs (`[Required]`, `[Range]`, etc.).
  Exemples :
- Vérifier l'unicité d'un e-mail.
- Contrôler la cohérence entre deux champs (ex. `StartDate < EndDate`).
- Valider un format spécifique (numéro de série, code client, etc.).

```
var user = new User { Email = "test", StartDate = DateTime.Today, EndDate =
DateTime.Today.AddDays(-1) };
var errors = user.Validate();
```

Validation côté MongoDB (Schema Validator)

- `validator` → définit les règles
- `$jsonSchema` → structure du document
- `validationAction` → comportement en cas d'erreur
- `validationLevel` → portée du contrôle

Validateur via C#

- validateur via C#

```
var schema = new BsonDocument
{
  { "bsonType", "object" },
  { "required", new BsonArray { "name", "email", "age", "createdAt"
} },
  { "properties", new BsonDocument {
      { "name", new BsonDocument { { "bsonType", "string" }, {
"minLength", 2 } } },
   { "email", new BsonDocument { { "bsonType", "string" }, {
"pattern", "^[^@\\s]+@[^@\\s]+\\.[^@\\s]+$" } } },
      { "age", new BsonDocument { { "bsonType", "int" }, {
"minimum", 18 }, { "maximum", 120 } } },
      { "createdAt", new BsonDocument { { "bsonType", "date" } } }
  }}
};
```



CAP COMPÉTENCES
Activateur de Talents

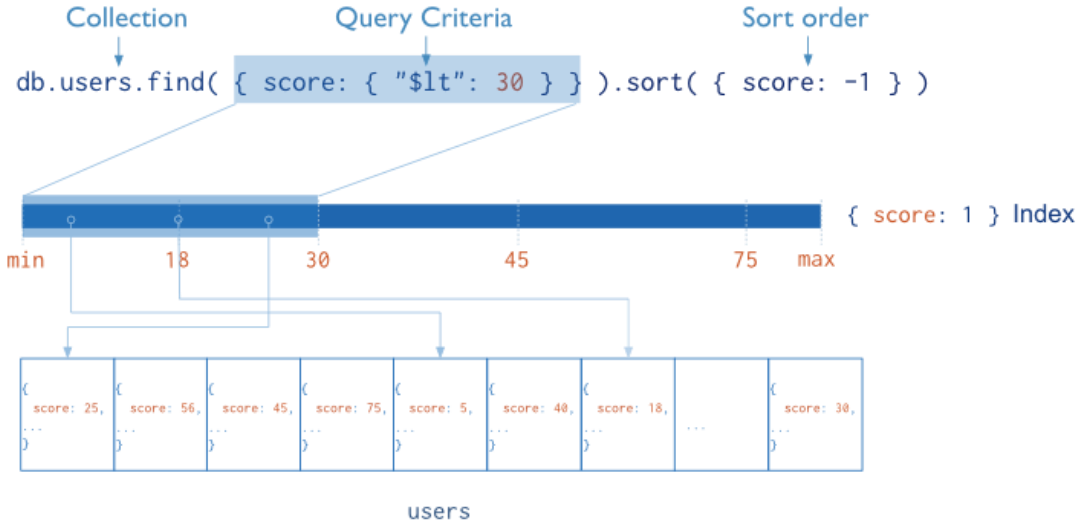
EURO INFORMATION

- L’indexation sous MongoDB

Chapitre 3

Les index

- Les index permettent un exécution efficace des requêtes dans MongoDB
- Sans index, MongoDB parcourt tous les documents d'une collection pour répondre à une requête. Avec un ou plusieurs index appropriés, le nombre de documents à inspecter diminue.
- Un index stocke les valeurs d'un ou plusieurs champs dans l'ordre des valeurs.
- Ainsi les requêtes
  par égalité de valeurs,
  par plage de valeurs
  et les tris sont optimisés.



Index par défaut et création d'index

- MongoDB crée par défaut un index sur le champ « _id ». Celui-ci est de type « unique » pour s'assurer que deux documents n'ont pas le même identifiant.

- Pour créer un index, on utilise la méthode suivante via le shell :

```
db.<collection>.createIndex( <key and index type specification>, <options> )
```

  - <key> : un document précisant le ou les champs support de l'index et leurs types
  - <options> : un document précisant certaines options

- Exemples :

```
db.produits.createIndex({"_id" : "hashed"})
db.produits.createIndex({"prix" : 1})
```

Types d'index

- **Un seul champ** : le type d'index le plus simple, ex:

```
db.produits.createIndex({"prix" : 1})
```

  - (le sens a peu d'importance car MongoDB peut parcourir ces index dans les deux directions pour un tri)

- **Index composé** : s'appuie sur plusieurs champs, ex:

```
db.users.createIndex({ userid: 1, score: -1 })
```

  - (le sens est important: les tris { userid: 1, score: -1 } et { userid: -1, score: 1 } pourront utiliser l'index mais pas les tris { userid: 1, score: 1 } ou { userid: -1, score: -1 })

- **Index multivalué** : s'appuie sur le contenu d'un tableau, ex:

  -
```
db.users.createIndex({ addr.zip: 1 })
```
    Exemple de document :

  -
```
{userid:1, addr:[{zip:"75017",…},{zip:"75020",…}] }
```
    (MongoDB va créer une entrée dans l'index par valeur dans le tableau)

Types d'index

- **Index geospatial** : s'appuie sur des coordonnées GPS, ex:

```
db.places.createIndex({ loc : "2dsphere" })
```

(permet d'utiliser l'opérateur `$geonear` pour faire des recherches geospatiales)

- **Index texte** : s'appuie sur un ou plusieurs champs texte d'une collection, ex:

```
db.reviews.createIndex({ title: "text", description: "text" },
 { weights:{title:5}, default_language: "french" })
```

(permet d'utiliser l'opérateur `$text` pour faire des recherches textuelles)
(il ne peut y avoir qu'un index de type texte par collection : la recherche textuelle se fait sur l'ensemble des colonnes de l'index)

- **Index hashés** : stocke des hashs de valeurs en index, ex:

```
db.users.createIndex({ "_id": "hashed" })
```

(support des clés de sharding hashés)

Propriétés d'index

- **Unique** : rejette les valeurs dupliquées sur une valeur (index à un seul champ) ou sur une combinaison de valeurs (index composé)

```
db.members.createIndex( { "user_id": 1 }, { unique: true } )
```

(un seul document autorisé avec le champ manquant, c.a.d. null)

- **Clairsemé** : indexe uniquement les documents ayant le champ renseigné, ou au moins un des champs renseignés pour un index composé.

```
db.addresses.createIndex( { "xmpp_id": 1 }, { sparse: true } )
```

(combiné avec « unique » : permet plusieurs documents n'ayant pas le champ renseigné)

- **Partiel** (>3.2) : indexe uniquement les documents répondant à certaines conditions exprimées par un filtre. Opérateurs possibles: `$eq, $exists, $gt, $gte, $lt, $lte, $type, $and` (au premier niveau)

```
db.restaurants.createIndex( { rating:1, cuisine: 1, name: 1 },
    { partialFilterExpression: { rating: { $gt: 5 } } } )
```

(économise de l'espace de stockage et du temps de calcul)

Propriétés d'index

- **TTL** : permet de supprimer des documents d'une collection après un certain temps

```
db.eventlog.createIndex( { "lastModifiedDate": 1 },
    { expireAfterSeconds: 3600 } )
```

*(Si **le** champ est un tableau, prend la date la plus ancienne. Si **le** champ est vide ou n'est pas une date, pas d'expiration)*
(Peut être intéressant pour des logs ou des sessions par exemple)

- **Nom personnalisé** :

```
db.eventlog.createIndex( { ... }, { name: "lenom
" } )
```

(par défaut, MongoDB concatène les noms et sens de tous les champs de l'index pour établir son nom)

Wildcard index

- Les index « joker » permettent de supporter des requêtes sur des champs non connus d'avance. Ex:

```
{ "userMetadata" : { "likes" : [ "dogs", "cats" ] } }
{ "userMetadata" : { "dislikes" : "pickles" } }
{ "userMetadata" : { "age" : 45 } }
{ "userMetadata" : "inactive" }
```

- La syntaxe :

```
db.userData.createIndex( { "userMetadata.$**" : 1 } )
```

- Il est possible d'exclure certains champs :

```
db.collection.createIndex(
   { "$**" : 1 },
   { "wildcardProjection" :
     { "fieldA" : 0, "fieldB.fieldC" : 0 }
   }
)
```

Wildcard index

- Les index « joker » :
  - Ne peuvent prendre en charge qu'une condition de filtre.
  - Ne peuvent pas être combinés avec un autre index « joker » sur une même requête.
  - Sont « clairsemé » : pas utilisable pour rechercher une absence de champ/valeur.
  - Ne peuvent servir de clé de shard.
  - Ne peuvent pas être composé.
  - Ne peuvent pas être affublés des propriétés « TTL » et « Unique ».
  - Ne peuvent pas être de type « 2d/2dSphere » et « hashed »

- Les index « joker » de type « text » sont différents : ils indexent toutes les données textuelles trouvées dans un document :

```
db.collection.createIndex( { "$**": "text" } )
```

Index « timeseries » (>= 5.0)

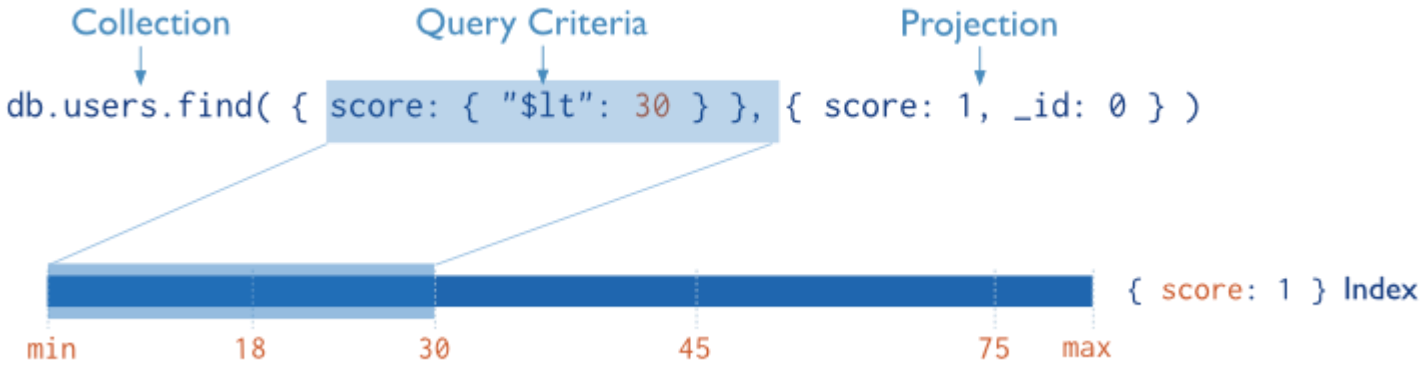
- Les index sur les collections « `timeseries` » :
  - Ne peuvent être créé que sur les champs déclarés « `timeField` » et « `metaField` » (ou si sous-document de métadonnées, sur les champs de celui-ci).
  - Pour le champs déclaré « `metaField` », ne supporte pas les index de type « `2d` », « `2dsphere` » et « `text` » (=5.0)
  - Pour le champs déclaré « `metaField` », ne supporte pas les index de type « `text` » (>= 5.1)
  - Ne supportent pas le propriétés « `TTL` », « unique » et « partial ».

- Avec le version 6.0, des améliorations importantes ont été apportées :
  - Possibilité de créer un index composé sur les champs déclarés « `timeField` », « `metaField` » et sur les champs contenant les mesures.
  - Possibilité de créer un index de type « `partial` » et « `2dsphere` » sur n'importe quel champs.
  - Possibilité d'utiliser les opérateurs « `$or` », « $in » et « `$geoWithin` » sur un index « `partial` ».

Requêtes couvertes

- Une requête est dite couverte lorsqu'elle n'inclut dans sa projection et dans son filtre de requête que des champs d'un index.

- Dans ce cas, MongoDB retourne ses résultats depuis l'index sans scanner de documents ➔ grande efficacité.



(dans l'exemple, la projection doit explicitement exclure le champ « _id » pour que la requête soit couverte, car l'index n'inclut pas le champ « _id »)

Intersection d'index

- MongoDB peut utiliser une intersection d'index pour répondre aux requêtes.
- Par exemple, si une collection « orders » a pour index { qty: 1 } et { item: 1 }, la requête suivante utilisera une intersection de ces index :

```
db.orders.find( { item: "abc123", qty: { $gt: 15 } } )
```

- Un index composé peut être utilisé partiellement (par son début).
- Par exemple, si une collection « orders » a pour index { qty: 1 } et { status: 1, ord_date: -1 }, la requête suivante utilisera une intersection de ces index :

```
db.orders.find( { qty: { $gt: 10 } , status: "A" } )
```



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

L'indexation avec C#

Chapitre 4

Intersection d'index

- permet d'accélérer les recherches sur un seul champ
- un index permet de localiser les documents plus rapidement.

```
var collection = db.GetCollection<CompteBancaire>("comptes");
```

- Index croissant

```
var indexKeys = Builders<CompteBancaire>.IndexKeys.Ascending(c => c.NumeroCompte);

await collection.Indexes.CreateOneAsync(
    new CreateIndexModel<CompteBancaire>(indexKeys)
);
```

Création d'index

- Création d’un index :

```
var indexKeys = Builders<CompteBancaire>.IndexKeys.Ascending(c => c.ClientId);
var options = new CreateIndexOptions { Name = "idx_client_id" };

await collection.Indexes.CreateOneAsync(
    new CreateIndexModel<CompteBancaire>(indexKeys, options)
);
```

Index multiclés (MultiKey Index)

- Un index multiclés est automatiquement créé lorsqu'un champ indexé contient un `tableau`.
  MongoDB indexe chaque élément du tableau, ce qui permet de rechercher rapidement des documents contenant une valeur spécifique dans un champ de type liste.

```
var collection = db.GetCollection<BankAccount>("accounts");
```

- Chaque élément du tableau Beneficiaries sera indexé

```
var indexKeys = Builders<BankAccount>.IndexKeys.Ascending(a =>
a.Beneficiaries);

await collection.Indexes.CreateOneAsync(
    new CreateIndexModel<BankAccount>(indexKeys)
);
```

Index composé

- Un index composé porte sur plusieurs champs.

```
var collection = db.GetCollection<Transaction>("transactions");

var indexKeys = Builders<Transaction>.IndexKeys
    .Ascending(t => t.ClientId)
    .Descending(t => t.DateOperation);

await collection.Indexes.CreateOneAsync(
    new CreateIndexModel<Transaction>(indexKeys)
);
```

Index unique

- Empêche l'insertion de doublons sur un champ critique

```
var collection = db.GetCollection<BankAccount>("accounts");

var indexKeys = Builders<BankAccount>.IndexKeys.Ascending(a =>
a.AccountNumber);
var options = new CreateIndexOptions { Name = "idx_unique_account_number",
Unique = true };

await collection.Indexes.CreateOneAsync(
    new CreateIndexModel<BankAccount>(indexKeys, options)
);
```

Lister / Supprimer un index

- Les index peuvent être consultés et supprimés directement depuis le code C#.
  Cela permet de :

- Vérifier les index existants d'une collection.

- Supprimer ceux qui sont obsolètes ou inutilisés pour améliorer les performances.

- Lister les index d'une collection :

```
var indexes = await collection.Indexes.ListAsync();
await indexes.ForEachAsync(index => Console.WriteLine(index));
```

Lister / Supprimer un index

- Supprimer un index spécifique

```
await collection.Indexes.DropOneAsync("idx_unique_account_number");
Supprimer tous les index :
await collection.Indexes.DropAllAsync();
```

Index partiel

- Un index partiel ne concerne qu'une partie des documents d'une collection.

```
    var keys = Builders<BankAccount>.IndexKeys.Ascending(a =>
    a.AccountNumber);

    var options = new CreateIndexOptions
    {
        Name = "idx_active_accounts",
        PartialFilterExpression =
    Builders<BankAccount>.Filter.Eq(a => a.IsActive, true)
    };

    await collection.Indexes.CreateOneAsync(
        new CreateIndexModel<BankAccount>(keys, options)
);
```

Index TTL

- Un index TTL (Time-To-Live) permet de supprimer automatiquement les documents après un certain délai.

```
var indexKeys = Builders<Session>.IndexKeys.Ascending(s => s.ExpirationDate);

var options = new CreateIndexOptions
{
    Name = "idx_ttl_session_expiration",
    ExpireAfter = TimeSpan.Zero // suppression immédiate après ExpirationDate
};

await collection.Indexes.CreateOneAsync(
    new CreateIndexModel<Session>(indexKeys, options)
);
```



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

L'optimisation des requêtes et le profiling

Chapitre 5

Stratégies d'indexation

- Pour définir les meilleurs index pour une application, il faut prendre en compte un certain nombre de facteurs :
  - Les requêtes attendues
  - La proportion de lectures par rapport aux écritures
  - La quantité de mémoire disponible sur le système
- Au moment de définir sa stratégie d'indexation, il faut avoir une bonne vision des requêtes effectuées par l'application (filtres de recherches, projection, tri).
- La création et la maintenance d'index ont un coût en terme de temps de calcul, mais se justifient pour des requêtes fréquentes sur de larges ensembles de données.
- La fréquence relative d'une requête permet aussi de déterminer si un index se justifie.
- Tester sur une copie des données de production plusieurs stratégies d'index est une bonne idée pour voir celle qui fonctionne le mieux.

Créer des index pour supporter ses requêtes

- Un index prend en charge une requête quand l'index contient tous les champs scannés par la requête. Ainsi MongoDB n'a pas besoin de scanner la collection ➔ performance.

- Si toutes les requêtes utilisent le même champ, créez un index avec ce seul champ.

- Si les requêtes utilisent un champ et parfois un deuxième champ en conjonction avec le premier, créez un index composé de ces deux champs (dans le même ordre).
  - Par exemple un index `{ x: 1, y: 1, z: 1 }` permet de supporter des requêtes qui feraient appel aux index
    `{ x: 1, y: 1 }` et `{ x: 1 }`

- Un index composé peut aussi supporter un filtre puis un tri.
  - Exemple un index `{ x: 1, z: 1 }` supporte la requête

```
db.collection.find( { x: 5 } ).sort( { z: 1
```

mais pas

```
db.collection.find( { z: 10} ).sort( { x: 1
```

Index et tri des requêtes

- Dans MongoDB, les opérations de tri peuvent obtenir l'ordre des documents en se basant sur l'ordre d'un index ➔ meilleurs performances.

- Si aucun index n'est utilisable, le tri se fait en mémoire.
  (Une opération de tri est avortée si elle utilise 100 Mo de mémoire sauf allowDiskUse())

- Un index avec un seul champ supporte les tris dans les deux directions.

- Un index composé supporte les tris dans le même ordre de colonne et dans les deux directions globales.
  - Par exemple, un index { a: 1, b: 1 } supporte le tri { a: 1, b: 1 } mais pas le tri { b: 1, a: 1 }
  - Par exemple, un index { a: 1, b: -1 } supporte les tris { a: 1, b: -1 } et { a: -1, b: 1 }, mais pas le tri { a: -1, b: -1 }

- Un index composé peut être utilisé partiellement pour faire un tri
  - Par exemple, un index { a: 1, b: 1, c:1 } supporte les tris { a: 1, b: 1 } et { a: 1 }

S’assurer que les index « tiennent » en RAM

- Pour de meilleurs performances, assurez-vous que les index « tiennent » entièrement en RAM pour éviter les accès disques.
- Pour connaître la taille des index d’une collection, utilisez la commande :

```
db.<collection>.totalIndexSize()
```

- Si vous avez plusieurs collections, assurez-vous que l’ensemble de leurs index « tiennent » en RAM
- Assurez-vous également d’avoir de la place en RAM pour le jeu de résultat de travail des requêtes.

Optimisation des requêtes

- Les index sont la base pour une optimisation des requêtes.
- La sélectivité d'une requête entre également en jeu : une requête sélective permet de limiter le nombre de documents analysés / retournés.
- Restreindre le nombre de résultats pour limiter le trafic réseau :

```
db.posts.find().sort( { timestamp : -1 } ).limit(1
```

- Utiliser la projection pour récupérer uniquement les données nécessaires.
- Utiliser la méthode « hint » pour suggérer l'utilisation d'un index :

```
db.users.find({nom:"DUPONT", age:{$gt:50}}).hint( { age: 1 } )
```

Profiler de base de données

- Le profiler collecte des données détaillées sur les opérations d'écritures, curseurs, commandes adressées aux instances de MongoDB.
- Le profilage peut être activé par base de données ou par instance, via le shell ou via les drivers.
- Un niveau de profilage est sélectionnable :
  - 0 : profiler désactivé, pas de collecte de données. Les requêtes lentes sont écrites en log.
  - 1 : collecte des données pour les requêtes lentes uniquement.
  - 2 : collecte des données pour toutes les opérations.
- On peut choisir le seuil en millisecondes pour qu'une requête soit considérée comme lente et un pourcentage des opérations à profiler (défaut = 1 ➔ 100%)
- Pour l'activation du profiling pour une base de données :

```
db.setProfilingLevel(<niveau>, <seuil>, <sampleRate>)
```

Profiler de base de données

- Pour connaître le niveau de profiling en cours sur une base de données :

```
db.getProfilingStatus()
```

- Pour activer le profiling au niveau d'une instance MongoDB, définir dans la configuration :
  - operationProfiling.slowOpThresholdMs : le seuil en millisecondes
  - operationProfiling.mode : le niveau de profiling

- Dans le cas d'un shard cluster, il faut activer le profiling sur chaque instance. (« mongos » n'est qu'un routeur).

- Les données collectées par le profiling sont écrites dans la collection « system.profile » de la base de données concernée. Cette collection est cappé à 1 Mo par défaut.

- Pour faciliter l'interprétation de ces données, il est possible de joindre à chaque requête un commentaire qui sera ajouté à ces données :

```
db.produits.find().comment("liste produits")
```



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

L’analyse des plans d’exécution

Chapitre 6

Explication des requêtes

- La fonction « explain() » permet de retourner un certain nombre d'informations sur une requête :
  - **queryPlanner** : le plan d'exécution choisi par l'optimiseur de requête, et la liste des plans rejetés
  - **executionStats** : détails sur l'exécution du plan vainqueur

- Le comportement de la fonction et le niveau de détail retourné dépendent du niveau de verbosité :
  - **queryPlanner** : MongoDB lance l'optimiseur de requête pour choisir le plan d'exécution vainqueur et retourne l'info « queryPlanner »
  - **executionStats** : MongoDB lance l'optimiseur de requête pour choisir le plan d'exécution vainqueur, l'exécute et retourne les infos « queryPlanner » et « executionStats »
  - **allPlansExecution** : MongoDB lance l'optimiseur de requête pour choisir le plan d'exécution vainqueur, l'exécute et retourne les infos « queryPlanner » et « executionStats ». Retourne aussi des statistiques d'exécution pour les plans rejetés.

```
db.<collection>.<requete>.explain([verbosity])
ex: db.produits.find({prix:{$gt:100}}).explain("executionStats")
```

Explication des requêtes

- Champs principaux du résultat (préfixe commun : **`explain.`**) :

Section « queryPlanner »	
`winningPlan`	Le plan d'exécution choisi par l'optimiseur
`winningPlan.stage`	Type d'étape : COLLSCAN (parcours collection), IXSCAN (index), FETCH (récupération), SHARD_MERGE…
`winningPlan.inputStage(s)`	Les étapes filles, exécutées juste avant
`rejectedPlans`	Les plans rejetés
Section « executionStats »	
`nReturned`	Nombre de documents retournés
`executionTimeMillis`	Temps d'exécution en millisecondes
`totalKeysExamined`	Nombre de clés d'index examinées
`totalDocsExamined`	Nombre de documents examinés
`executionStages(.inputStage…)`	Arbre des étapes exécutées — chaque étape porte ses propres totalKeysExamined / totalDocsExamined



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Fonctionnement de MongoDB : transaction, verrou

Chapitre 7

Transactions multi-documents

- A partir de la version 4.0, il est possible de faire des transactions regroupant des modifications de plusieurs documents.

- Cela fonctionne sur les ReplicaSet (>= 4.2 : sur les clusters de shard).
- Les transactions sont associées à une session qu'il faut passer à chaque opération de la transaction :

```
session = db.getMongo().startSession( { readPreference: { mode: "primary" } } );
session.startTransaction( { readConcern: { level: "snapshot" }, writeConcern: { w: "majority" }
);
try {
   employeesCollection.updateOne( { employee: 3 }, { $set: { status: "Inactive" } } );
   eventsCollection.insertOne( { employee: 3, status: { new: "Inactive", old: "Active" } } );
} catch (error) {
   session.abortTransaction();
   throw error;
}
session.commitTransaction();
session.endSession();
```

Verrou

- Les écritures sur un document sont atomiques : tout ou rien, jamais de vue incohérente pour les clients.
- MongoDB utilise un verrouillage à plusieurs niveaux : les opérations peuvent obtenir un verrou global, sur une base ou une collection.
- Les moteurs de stockage peuvent implémenter des verrous plus fins (par exemple, au niveau document dans WiredTiger).
- MongoDB utilise des verrous en lecture et écriture, ce qui peut permettre un accès en lecture même si un verrou en écriture a lieu sur la ressource.
- Quatre modes : partagé S (lecture), exclusif X (écriture), et leurs intentions IS / IX qui signalent un verrou plus fin à venir (S ou X à un niveau ➔ IS / IX aux niveaux supérieurs).

Verrou

- Par exemple, lors du verrouillage d'une collection pour l'écriture (en utilisant le mode X), le verrou de base de données et le verrou global sont verrouillés dans le mode exclusif intentionnel (IX).

- Une base de données peut être verrouillée simultanément en mode IS et IX, mais un verrou exclusif (X) ne peut coexister avec aucun autre mode, et un verrou partagé (S) ne peut coexister qu'avec des verrous partagés (IS).

- Les demandes de verrous sont mis en file d'attente dans leur ordre d'arrivée. Toutefois pour optimiser les performances, lorsqu'une demande est accordée, toutes les demandes compatibles sont accordées.

- Par exemple, avec la file IS ➔ IS ➔ X ➔ X ➔ S ➔ IS, MongoDB accordera les 2 premiers IS mais aussi les autres S et IS. Lorsqu'ils auront été tous relâchés, les X seront accordés.

Verrou

- Pour la plupart des opérations de lecture et d'écriture, le moteur de stockage WiredTiger utilise une gestion optimiste de la concurrence : il utilise des verrous d'intentions au niveau global, base et collections.

- Si un conflit est détecté entre deux opérations, une d'entre elles sera retentée plus tard de manière transparente.

- Certaines opérations impliquant plusieurs bases requièrent un verrou global. D'autres, comme la commande d'administration « collMod », requièrent un verrou exclusif sur la base.

- Dans un cluster de shard, chaque shard gère ses propres verrous.
  Dans un ReplicaSet, le primaire gère ces verrous. Les secondaires rejouent le journal d'opérations dans l'ordre.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Transaction en C#

Chapitre 8

Transaction avec C#

- Ensemble d'opérations exécutées comme une seule unité logique.

Soit toutes les opérations réussissent, ne soit aucune n'est appliquée.
Cohérence des données.

- `IClientSessionHandle` (Session)

```
using var session = await client.StartSessionAsync();
```

- Nécessaire pour regrouper plusieurs opérations dans une même transaction.

- Lance une nouvelle transaction à l'intérieur de la session.
- Toutes les opérations suivantes seront exécutées de façon atomique.

```
session.StartTransaction();
```

Transaction avec C#

- Opérations dans la transaction
- Les opérations (`InsertOne, UpdateOne, DeleteOne,` etc.) doivent recevoir le paramètre session :

```
await collection.UpdateOneAsync(session, filter, update);
```

```
CommitTransactionAsync()
```

- Valide toutes les opérations si aucune erreur ne s'est produite.
- Les modifications sont définitivement écrites dans la base.

```
await session.CommitTransactionAsync();
```

Transaction C#


```
AbortTransactionAsync()
```

- Annule la transaction si une erreur survient (rollback).
- Aucune modification n’est appliquée à la base.

Transaction C#





CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

INTRODUCTION À LA RÉPLICATION ET AU SHARDING

MODULE 5

Introduction à la réplication et au Sharding

- Réplication
  - Introduction
  - La réplication Primary-Secondary
  - Fonctionnement de la bascule automatique du nœud Primary
  - Membres secondaires particuliers
  - Mise en place de la réplication

- Sharding
  - Introduction
  - Le sharding avec MongoDB
  - Mise en place du sharding
  - Répartition manuelle des données, ShardTag, TagRange



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Réplication : introduction

Chapitre 1

Réplication

- Disponible depuis la version 1.6, c'est une méthodologie pour synchroniser des données sur plusieurs serveurs.
- Permet la redondance des données et leurs hautes-disponibilités (permet la perte d'un serveur).
- Il est possible de répartir les ordres de lectures sur les différents serveurs (mais ce n'est pas toujours une bonne idée).
- Il est également possible d'ajouter des copies pour des besoins spécifiques :
  - « hidden » pour du reporting
  - « delayed » pour pouvoir faire un retour arrière après une mise en production



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

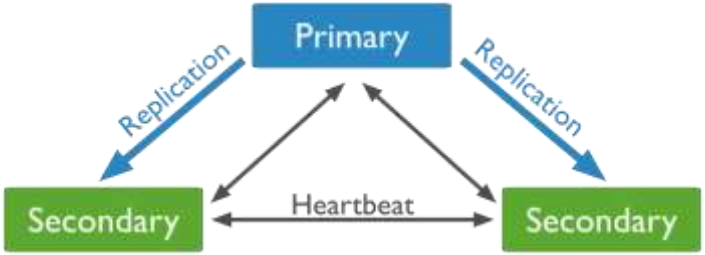
La réplication Primary-Secondary

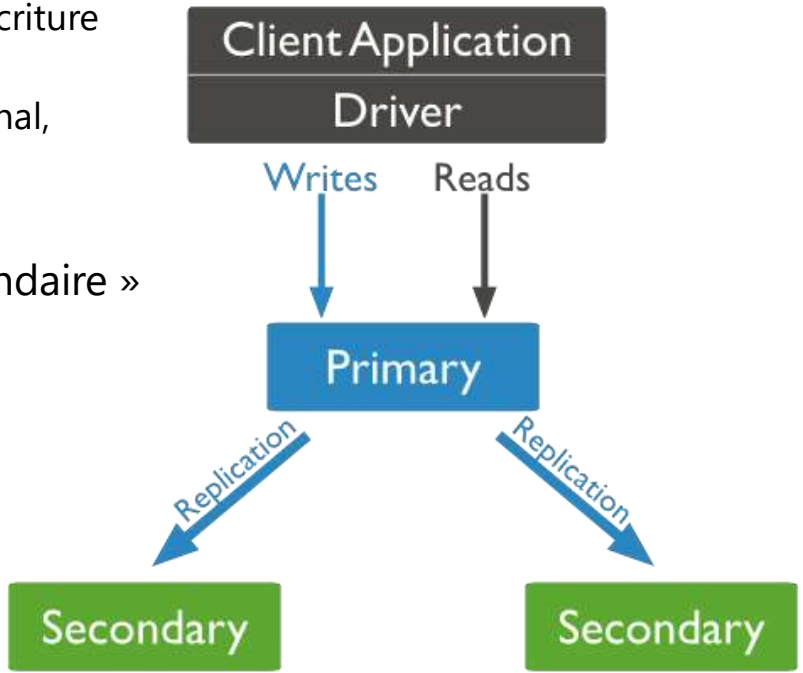
Chapitre 2

La réplication Primary-Secondary

- Un « ReplicaSet » est un groupe d'instances du programme « mongod » qui maintiennent le même ensemble de données.
- Un ReplicaSet contient :
  - Un nœud « primaire » qui reçoit les requêtes en écriture et les écrit dans son journal d'opération
  - Des nœuds « secondaires » qui répliquent ce journal, et rejouent les opérations sur leurs instances

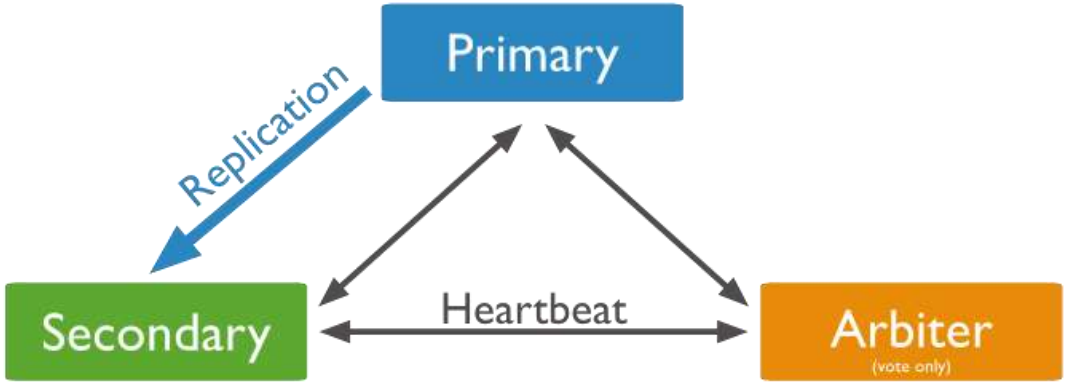
- Si le « primaire » devient inaccessible, un « secondaire » sera élu « primaire »





La réplication Primary-Secondary

- Il est possible d'ajouter dans le ReplicaSet une instance de « mongod » comme « arbitre ».
- Un « arbitre » ne maintient pas l'ensemble de données mais participe seulement aux élections.
- Cela peut être un moyen économique pour éviter de déployer deux secondaires, tout en maintenant un total de 3 votants aux élections.





CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

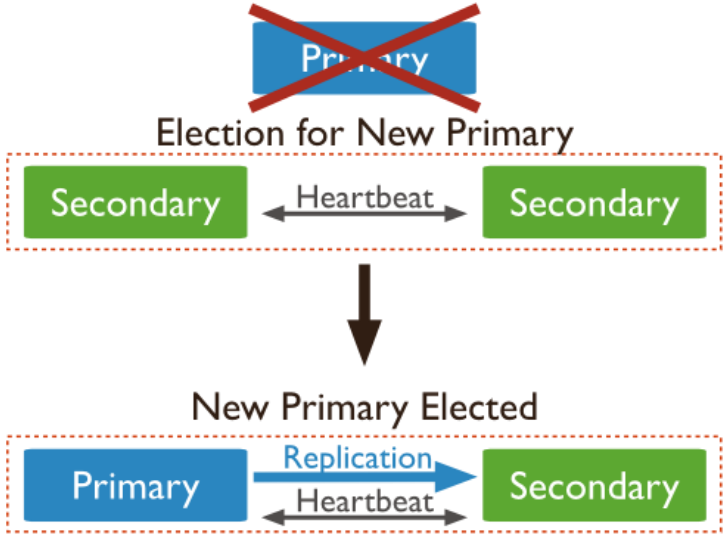
Fonctionnement de la bascule automatique du nœud Primary

Chapitre 4

Bascule automatique du nœud Primary

- Si le nœud « Primary » ne communique pas avec les autres du ReplicaSet pendant plus de 10 secondes, un nœud « secondaire » déclenche une élection.

- Le premier « secondaire » qui a déclenché l'élection et qui a reçu le plus de votes devient « primaire ».





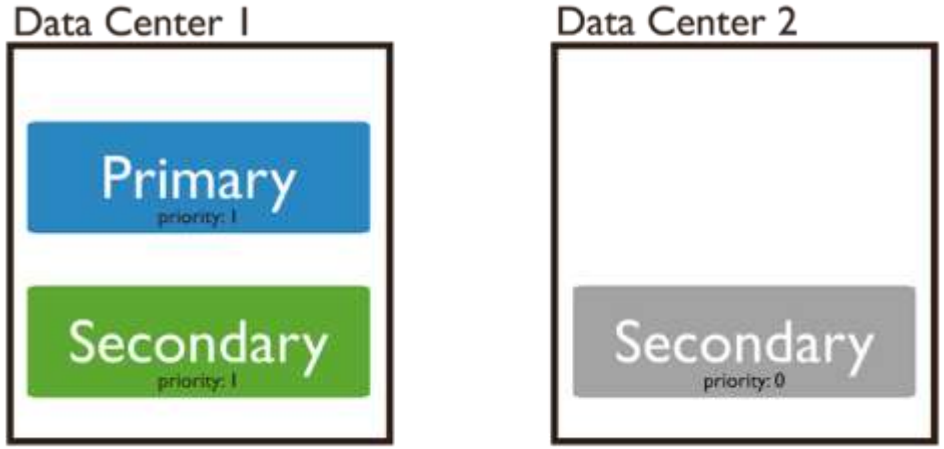
Membres secondaires particuliers

Chapitre 5

Membres à priorité nulle

- Un membre avec une priorité de 0 ne peut devenir primaire et ne peut déclencher d'élections.

- Peut être utile en cas de déploiement sur plusieurs datacenters, pour des serveurs de réserve par exemple.



Membres cachés

- Un membre avec une priorité de 0 et avec l'option « hidden : true » est invisible pour les clients et ne reçoit donc pas de lectures.

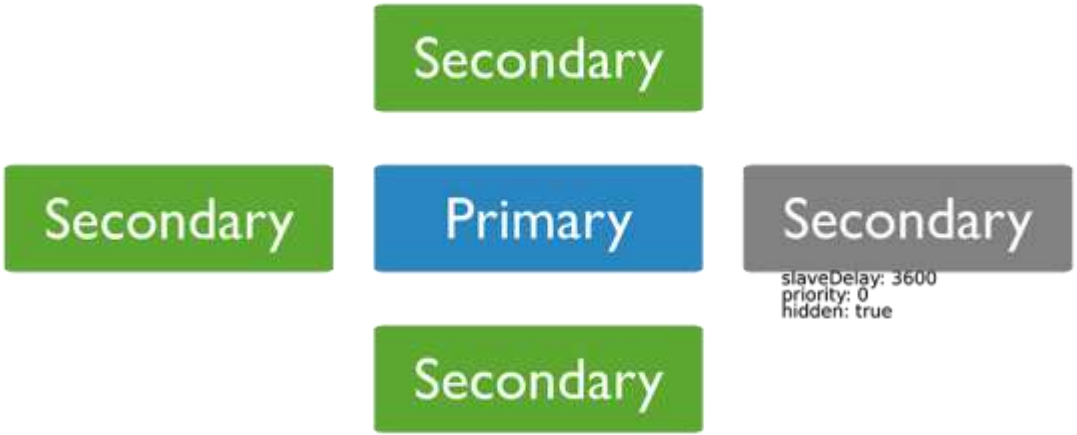
- Peut être utile pour des taches lourdes : reporting, backup, …



Membres retardés

- Un membre caché avec l'option « SecondaryDelaySecs » rejoue les opérations du journal avec le délai choisi.

- Peut être utile pour revenir en arrière en cas d'erreur humaine sur la base.



Membres non votants

- Un membre avec l'option « votes : 0 » ne participe pas aux votes : il peut y avoir jusqu'à 50 membres dans un ReplicaSet mais 7 votants maximum.

- Il doit aussi avoir une priorité de 0.





CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Mise en place de la réplication

Chapitre 6

Mise en place de la réplication

- Démarrer chaque membre du ReplicaSet avec l'information correspondante
  - Soit via la ligne de commande : `mongod --replSet "rs0"`
  - Soit via le fichier de configuration :

```
replication:
   replSetName: "rs0"
```

- Se connecter sur l'un des membres du ReplicaSet via le client **mongo**
- Initier le ReplicaSet dans le shell via la commande : `rs.initiate()`
- Vérifier la configuration initiale du ReplicaSet : `rs.conf()`
- Ajouter des membres dans le ReplicaSet : `rs.add("host:port")`
- Vérifier le statut du ReplicaSet : `rs.status()`



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Read Preference

Chapitre 7

Read Preference

- La préférence de lecture (Read Preference) détermine comment un driver MongoDB va adresser les opérations de lecture aux membres d'un Replica Set.

- Par défaut, un driver dirige les opérations de lecture au nœud primaire.

- Les différents modes sont:
  - « **primary** » : mode par défaut. Lit sur le nœud primaire.
  - « **primaryPreferred** » : Lit sur le nœud primaire. Si celui-ci est indisponible, lit sur les nœuds secondaires.
  - « **secondary** » : Lit sur les nœuds secondaires.
  - « **secondaryPreferred** » : Lit sur les nœuds secondaires. Si aucun nœud secondaire est disponible, lit sur le nœud primaire.
  - « **nearest** » : Lit sur le nœud ayant le moins de latence réseau, peu importe son type.

Read Preference

- Utiliser une préférence de lecture autre que « primary » peut être intéressant pour :
  - Récupérer des données dans un contexte non temps réel
  - S'adresser au nœud le plus « proche » si le ReplicaSet est déployé sur plusieurs datacenter (mode « nearest »)
  - Maintenir la disponibilité en cas d'incident sur le nœud primaire (mode « primaryPreferred »)

- On peut utiliser les modes « secondary » et « secondaryPreferred » pour augmenter la capacité de charge en lecture mais :
  - Tous les nœuds ont la même charge en écriture (replication)
  - La réplication est asynchrone (et pas synchronisée entre secondaires)
    Lire sur un secondaire risque de retourner des données obsolètes.
    Lire sur plusieurs secondaires risque de retourner des données différentes.
  - Si un nœud secondaire « tombe », les autres devront assurer la charge correspondante.

- Le sharding est une stratégie moins contraignante pour augmenter la capacité.

Consistance causale

- MongoDB permet aux développeurs de définir des relations de cause à effet entre les opérations envoyées à la base.
- Dans une session client sur un ReplicaSet, MongoDB va respecter ces relations pour exécuter les opérations dans le bon ordre.
- Cela permet de garantir par exemple de lire ses écritures après les avoir faites, et cela sur n'importe quel membre du ReplicaSet.
- Pas de garanties vis-à-vis des autres sessions.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

ReadConcern

Chapitre 8

ReadConcern

- Le paramètre « readConcern », évoqué lors des opérations de recherche, correspond au niveau de fraicheur des données lié à la propagation de celles-ci dans le ReplicaSet.

- Les principales valeurs possibles :
  - `local` : MongoDB va retourner la version la plus récente des données même si elle n'a pas été propagée sur une majorité de nœuds (valeur par défaut)
  - `majority` : garantit de récupérer la version des données la plus récente ayant été propagé sur une majorité de nœuds (nécessite le moteur de stockage WireTiger avec la version 1 du protocole de replication)

- « Majority » permet d'éviter de lire des données pouvant être « rollback » : écritures sur le primaire n'ayant pu être propagées avant un crash, rollback lors de son retour comme secondaire.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

WriteConcern

Chapitre 9

WriteConcern

- Le paramètre « writeConcern », évoqué lors des opérations d'insertion et de modification, correspond au niveau d'accusé de réception souhaité pour ces écritures.

```
{ w: <value>, j: <boolean>, wtimeout: <number> }
```

  - `w` : accusé de réception en mémoire sur les nœuds
    - 0 : pas d'accusé de réception
    - 1 : accusé de réception du « primaire » (par défaut)
    - n : accusé de réception de N nœuds (primaire compris)
    - majority : accusé de réception d'une majorité de nœuds (primaire compris)
  - `j` : accusé d'écriture dans le journal d'opération
    - vrai : accusé de réception du primaire (prend en compte « w » et attend accusé du même nombre de nœuds) (faux par défaut sauf si `w:majority`)
  - `wtimeout` : temps limite en ms pour l'atteinte des niveaux
    - Applicable si w > 1
    - Si le temps est dépassé, une erreur est retournée
    - Si aucune valeur définie (ou =0) et que les niveaux sont inatteignables, l'écriture reste bloquée indéfiniment



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Flux de changement

Chapitre 10

Flux de changement

- Les flux de changement permettent aux applications de recevoir en temps réel les changements de données.
- Une application peut souscrire aux changements sur une collection, une base ou un serveur entier. (sur un ReplicaSet ou un cluster de shard avec le moteur de stockage WiredTiger)
- On peut déclarer un pipeline d'agrégation pour filtrer et transformer les évènements.
- On peut également préciser depuis quel point du temps reprendre l'écoute.

```
using var cursor = await collection.WatchAsync();
await cursor.ForEachAsync(change =>
Console.WriteLine($"{change.OperationType} : {change.FullDocument}"));
```



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Sharding : introduction

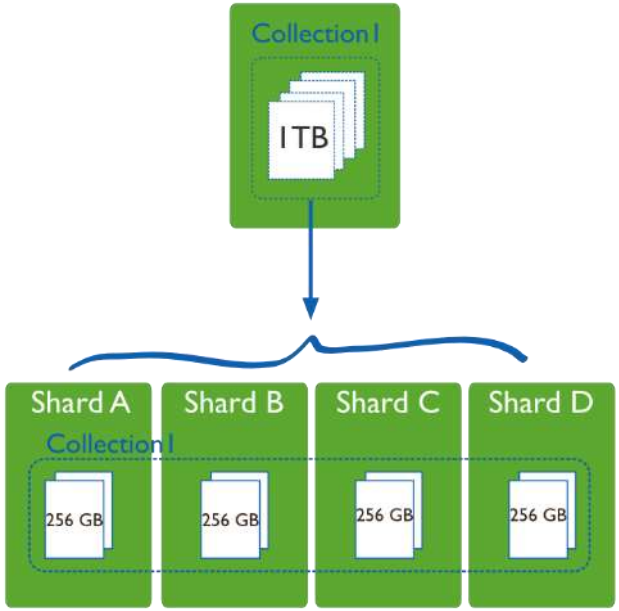
Chapitre 11

Sharding : introduction

- Méthode pour stocker les données des bases sur plusieurs machines.
- Les bases de données ayant de très forts volumes de données et de requêtes peuvent atteindre les limites des machines sur lesquelles elles se situent.
- Pour palier à cela, la première possibilité est la **scalabilité verticale** :
  - Consiste à ajouter plus de ressources CPU, RAM, stockage sur un serveur…
  - Toutefois les gros systèmes matériels sont beaucoup plus chers que les petits (non proportionnel).
  - Il y a également une limite dans la taille des instances que vous pouvez créer chez un fournisseur de cloud.

Sharding : introduction

- La deuxième possibilité est la **scalabilité horizontale** :
  - Consiste à répartir les données sur plusieurs serveurs appelés « shard » (et par conséquent les lectures et écritures sont également réparties).
  - Chaque shard est une base indépendante. L'ensemble des shards forment une base logique.





CAP COMPÉTENCES
Activateur de Talents

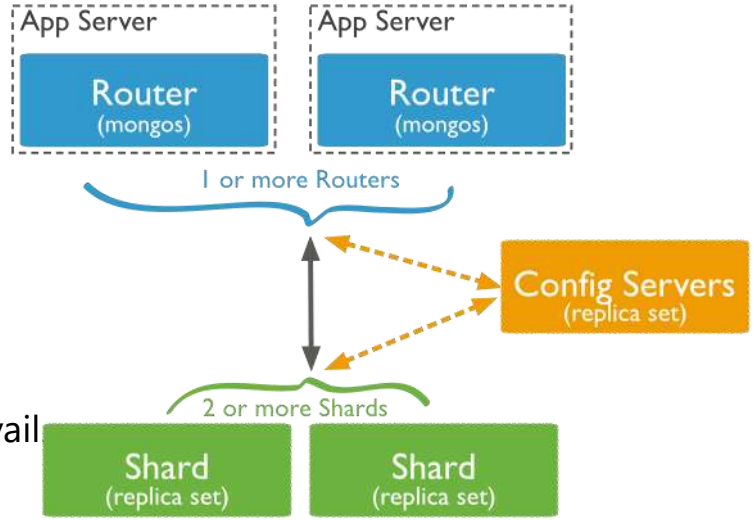
EURO INFORMATION

Le sharding avec MongoDB

Chapitre 12

Le sharding avec MongoDB

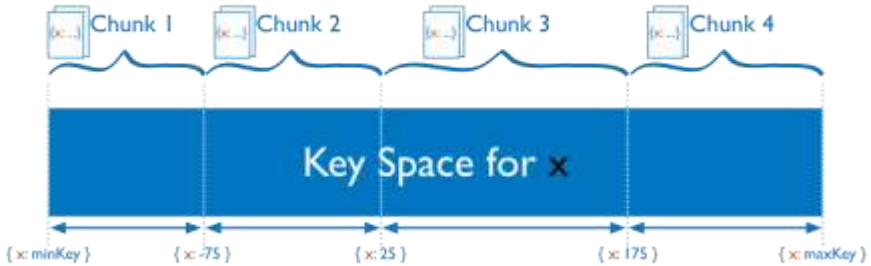
- MongoDB apporte le sharding via la mise en place d'un « sharded cluster » comportant :
  - Au moins 2 **Shard** stockant les données.
    (en ReplicaSet pour la haute disponibilité)
  - Au moins 1 **Routeur** (mongos) faisant interface entre les applis clients et les shards.
    Ils redirigent les requêtes vers les bons shards et retransmettent la réponse.
  - Des **Serveurs Config** stockant les métadonnées du cluster (mapping données – shard).
    Les routeurs utilisent cette base pour effectuer leur travail



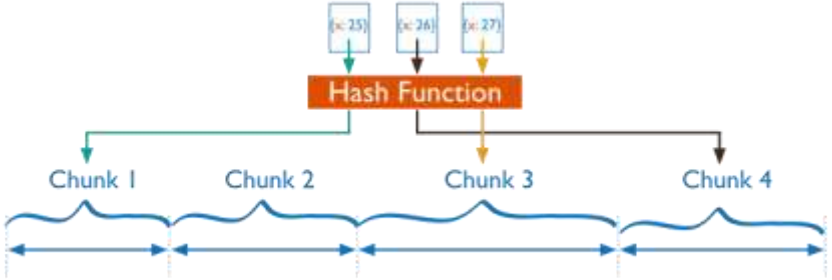
Le sharding avec MongoDB

- MongoDB distribue les données sur les shards par collection.
- La répartition se fait par une clé de shard (simple ou composé) qui doit, dans l'idéal, exister pour tous les documents de la collection.
- La répartition des valeurs de la clé sur les différents shards peut se faire de 2 manières :

par intervalles de valeur



par hash de valeur *(>= 2.4)*



Le sharding avec MongoDB

- Chaque méthode a ses avantages et ses inconvénients…

- La répartition par intervalles :
  - est plus efficace pour les requêtes utilisant ces intervalles
  - peut amener à une répartition déséquilibrée des « chunks » et donc des requêtes
  - choisir une clé de shard dont les valeurs augmentent progressivement dirigerait les écritures vers un seul shard (et potentiellement les lectures)

- La répartition par hash :
  - permet une répartition plus équitable car « aléatoire »
  - les requêtes utilisant des intervalles ne seront pas optimales
  - choisir une clé de shard avec peu de valeurs distinctes limite l'efficacité de la répartition



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Mise en place du sharding

Chapitre 13

Mise en place du sharding

- Afin de créer un « sharded cluster », on commence par déployer les serveurs de config sous la forme d'un ReplicaSet.
  - Tout d'abord on démarre les serveurs mongod :
    - Via la ligne de commande : `mongod --configsvr --replSet configReplSet --port <port> --dbpath <path>`
    - Ou via le fichier de configuration :

```
sharding:
   clusterRole: configsvr
replication:
   replSetName: configReplSet
```

  - Puis on initie le ReplicaSet :

```
rs.initiate( {
   _id: "configReplSet",
   configsvr: true,
   members: [
      { _id: 0, host: "<host1>:<port1>" },
      { _id: 1, host: "<host2>:<port2>" },
      { _id: 2, host: "<host3>:<port3>" }
   ]
} )
```

Mise en place du sharding

- Il faut ensuite démarrer les routeurs « mongos » :
  - Soit via la ligne de commande :

```
mongos --configdb
configReplSet/<cfgsvr1:port1>,<cfgsvr2:port2>,<cfgsvr3:port3>
```

  - Soit via la clé « `sharding.configDB` » de son fichier de configuration

- Mettre en place chaque shard sous la forme d'un ReplicaSet

- Ajouter les shards sur le cluster en vous connectant au routeur via le client « `mongo` »

```
sh.addShard( "rs1/mongodb0.example.net:27017" )
sh.addShard( "rs2/mongodb1.example.net:27017" )
```

Mise en place du sharding

- Activer ensuite le sharding au niveau de la base de données (< 6.0) :

```
sh.enableSharding("<database>")
```

- Créer les index qui seront les supports des clés de shard :

```
db. <collection>.createIndex(shard-key-pattern)
```

  - Ex1: `db.people.createIndex( { "zipcode": 1, "name": 1 } )`
  - Ex2: `db.alerts.createIndex( { "_id": "hashed" } )` *(!= index par défaut)*

- Sharder les collections :

```
sh.shardCollection("<database>.<collection>", shard-key-pattern)
```

  - Ex1: `sh.shardCollection("records.people", { "zipcode": 1, "name": 1 } )`
  - Ex2: `sh.shardCollection("events.alerts", { "_id": "hashed" } )`



La répartition manuelle des données

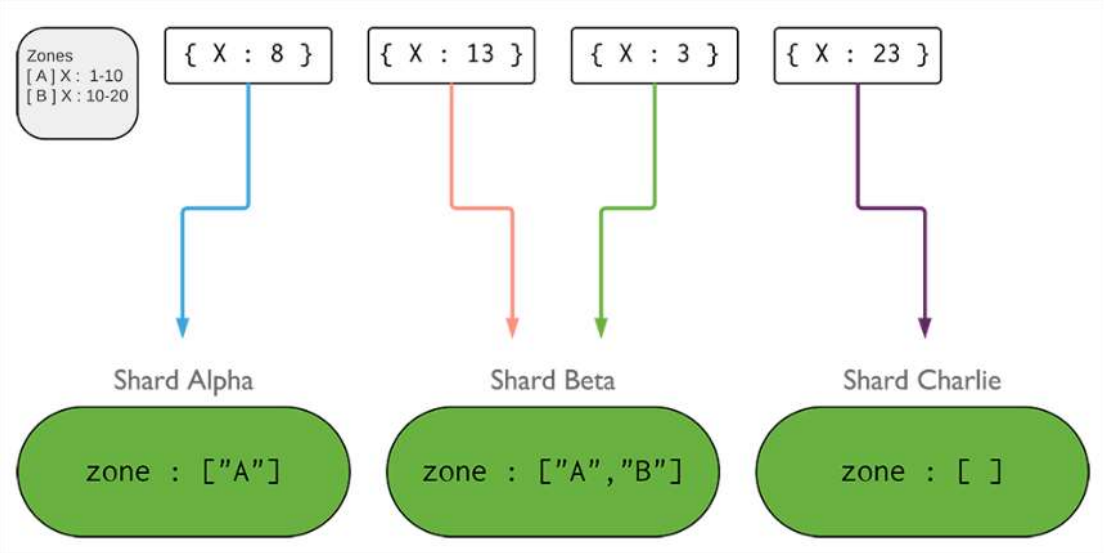
- Chapitre 14

Répartition manuelle des données

- Dans un cluster de shard, on peut créer des zones de données basées sur la clé de shard.
- Chaque zone peut être associée à un ou plusieurs shards.
- Les zones ne doivent pas se chevaucher en termes d'intervalles de valeurs.
- MongoDB s'occupe de migrer automatiquement les données selon les zones définies.
- Les zones peuvent être utiles pour :
  - Isoler un sous-ensemble de données sur un ensemble de shards donnée
  - Répartir les données géographiquement au plus proche des applications qui les consomment
  - Répartir les données selon les performances des shards

Répartition manuelle des données

- Exemple :



- Les intervalles de valeurs sont définis limite basse incluse et limite haute exclue.

- Le « balancer » va essayer de répartir les données équitablement selon les zones cibles, ou normalement si une donnée ne rentre dans aucun intervalle.

Répartition manuelle des données

- Les intervalles se définissent selon les champs de la clé de shard, utilisés hiérarchiquement si la clé est composée.

- Par exemple, pour des clés ayant la forme { a : 1, b : 2, c : 3 }, on peut définir :
  - { a : 1, b : 2, c : <mini>} ➔ { a : 1, b : 2, c : 500 } et { a : 1, b : 2, c : 500 } ➔ { a : 1, b : 2, c : <maxi> }
  - { a : 1, b : <mini> } ➔ { a : 1, b : 500 } et { a : 1, b : 500 } ➔ { a : 1, b : <maxi> }
  - { a : <mini> } ➔ { a : 500} et { a : 500 } ➔ { a : <maxi> }
  - { a : <mini>, b : <mini> } ➔ { a : 200, b : 500 }, { a : 200, b : 500 } ➔ { a : 300, b : <maxi> }, ...

- Avec une clé hashée, les intervalles s'expriment par valeur (avant hash) mais la répartition suit les valeurs hashées. ➔ comportement imprévisible ➔ à éviter

- Depuis la version 4.4, il est possible de combiner fields hashé et non hashé dans une clé :
  - { "fieldA" : 1, "fieldB" : "hashed" } : zones par (fieldA), réparties équitablement via le hash de (fieldB)
  - { "_id" : "hashed", "fieldA" : 1} : répartition par hash avec un champ (fieldA) cassant la croissance monotonique du hash (_id)

Répartition manuelle des données

- Les zones sont affectées au shards via l'instruction :

```
sh.addShardTag(<nom du shard>, <nom de zone>)
```

- Exemples :

```
sh.addShardTag("shard0000", "NYC")
sh.addShardTag("shard0001", "NYC")
sh.addShardTag("shard0002", "SFO")
sh.addShardTag("shard0002", "NRT")
```

- Les intervalles sont ensuite définis via l'instruction :

```
sh.addTagRange(<namespace>, <limite basse>, <limite haute>, <nom de zone>)
```

- Exemples :

```
sh.addTagRange("records.users", { zipcode: "10001" }, { zipcode: "10281" }, "NYC")
sh.addTagRange("records.users", { zipcode: "11201" }, { zipcode: "11240" }, "NYC")
sh.addTagRange("records.users", { zipcode: "94102" }, { zipcode: "94135" }, "SFO")
```



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

EXTENSION DE MONGODB

MODULE 6

Extension de MongoDB

- Les requêtes géo-spatiales
- L'Aggregation Framework
- Requêtes avancé avec L'Aggregation Framework



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Les requêtes géo-spatiales

Chapitre 1

Les requêtes géo-spatiales

- MongoDB permet d'effectuer des recherches basées sur des coordonnées géographiques (latitude, longitude) exprimées dans un espace sphérique.

- En se basant sur un index de type « 2dsphere », il est possible de faire des recherches sur différents objets géographique définis dans la norme GeoJSON : points, lignes, polygones, multi-points, multi-lignes, multi-polygones et une structure permettant de mélanger tous ces types.

- Les points GeoJSON sont exprimés en base sous la forme :

```
loc : { type: "Point", coordinates: [<longitude>, <latitude> ] }
```

- Les lignes GeoJSON sont exprimées en base sous la forme :

```
trip : { type: "LineString", coordinates: [[<longitude>, <latitude>], [<longitude>,
<latitude>], ...] }
```

Les requêtes géo-spatiales

- Les polygones GeoJSON sont exprimés en base sous la forme :

```
shape : { type: "Polygon", coordinates: [[<longitude>, <latitude>], [<longitude>,
<latitude>], ...] }
```

- Les multi-points GeoJSON sont exprimés en base sous la forme :

```
multi_loc : { type: "MultiPoint", coordinates: [[<longitude>, <latitude>], [<longitude>,
<latitude>], ...] }
```

- Les multi-lignes GeoJSON sont exprimées en base sous la forme :

```
multi_trip : { type: "MultiLineString", coordinates: [
  [[<longitude>, <latitude>], [<longitude>, <latitude>], ...],
  [[<longitude>, <latitude>], [<longitude>, <latitude>], ...],
  ...
] }
```

- Les multi-polygones GeoJSON sont exprimés en base sous la forme :

```
multi_shape : { type: "MultiPolygon", coordinates: [
  [[<longitude>, <latitude>], [<longitude>, <latitude>], ...],
  [[<longitude>, <latitude>], [<longitude>, <latitude>], ...],
  ...
] }
```

$nearSphere

- L'opérateur « $nearSphere » permet de ressortir des documents par ordre de proximité avec un point géographique déterminé (dans un espace sphérique).

- La syntaxe :

```
<GeoJSON object field>: {
     $nearSphere: {
       $geometry: {
          type: "Point" ,
          coordinates: [ <longitude> , <latitude> ]
       },
       $maxDistance: <distance in meters>,
       $minDistance: <distance in meters>
     }
   }
```

$geoWithin

- L'opérateur « $geoWithin » permet de ressortir des documents contenus dans une forme géométrique délimitant un espace géographique.

- La forme peut être un polygone, un multi-polygone, un rectangle ou un cercle.

  - Pour utiliser un polygone ou un multi-polygone GeoJSON :

```
<GeoJSON object field>: {
     $geoWithin: {
       $geometry: {
          type: <"Polygon" or "MultiPolygon"> ,
          coordinates: [ <coordinates> ]
       }
     }
   }
```

$geoIntersects

- L'opérateur « $geoIntersects » permet de ressortir des documents dont l'objet GeoJSON croise ou partage un côté avec une forme géométrique délimitant un espace géographique.
  - Par exemple avec un polygone GeoJSON :

```
<GeoJSON object field>: {
     $geoIntersects: {
       $geometry: {
          type: "Polygon" ,
          coordinates: [ <coordinates> ]
       }
     }
   }
```

Mongofiles

- Le programme « mongofiles » permet de manipuler des fichiers stockés en GridFS.

```
mongofiles <options> <commands> <filename>
```

  - **options** : options permettant de contrôler le comportement du programme
  - **commands** : l'action souhaitée
  - **filename** : le nom du fichier dans le système de fichier ou dans GridFS

- Les options :
  - `--host <hostname>` : l'hôte
  - `--port <port>` : le port
  - `--db <database>` : la base
  - `--type <mime>` : le type de fichier (action « put »)
  - `--replace` : remplace un fichier selon son nom plutôt que d'insérer un nouveau fichier avec le même nom
  - `--prefix <string>` : pour définir un autre préfixe (que « fs »)
  - `--local <fileName>` : pour distinguer le nom sur disque du nom dans GridFS

Mongofiles

- Les actions :
  - list <prefix> : liste les fichiers (optionnellement commençant par le préfixe)
  - search <string> : liste les fichiers dont le nom contient une chaîne
  - put <filename> : copie un fichier vers GridFS
  - get <filename> : récupère un fichier depuis GridFS
  - delete <filename> : supprime un fichier dans GridFS
  - get_id "<ObjectId>" : récupère un fichier selon son identifiant
  - delete_id "<ObjectId>" : supprime un fichier selon son identifiant

- Exemples :

```
mongofiles -d records list                    // liste les fichiers
mongofiles -d records search corinth          // cherche des fichiers
mongofiles -d records put 32-corinth.lp       // stocke un fichier
mongofiles -d records get 32-corinth.lp       // récupère un fichier
mongofiles -d records delete 32-corinth.lp    // supprime un fichier
```



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

L'Aggregation Framework

Chapitre 2

L’Aggregation Framework

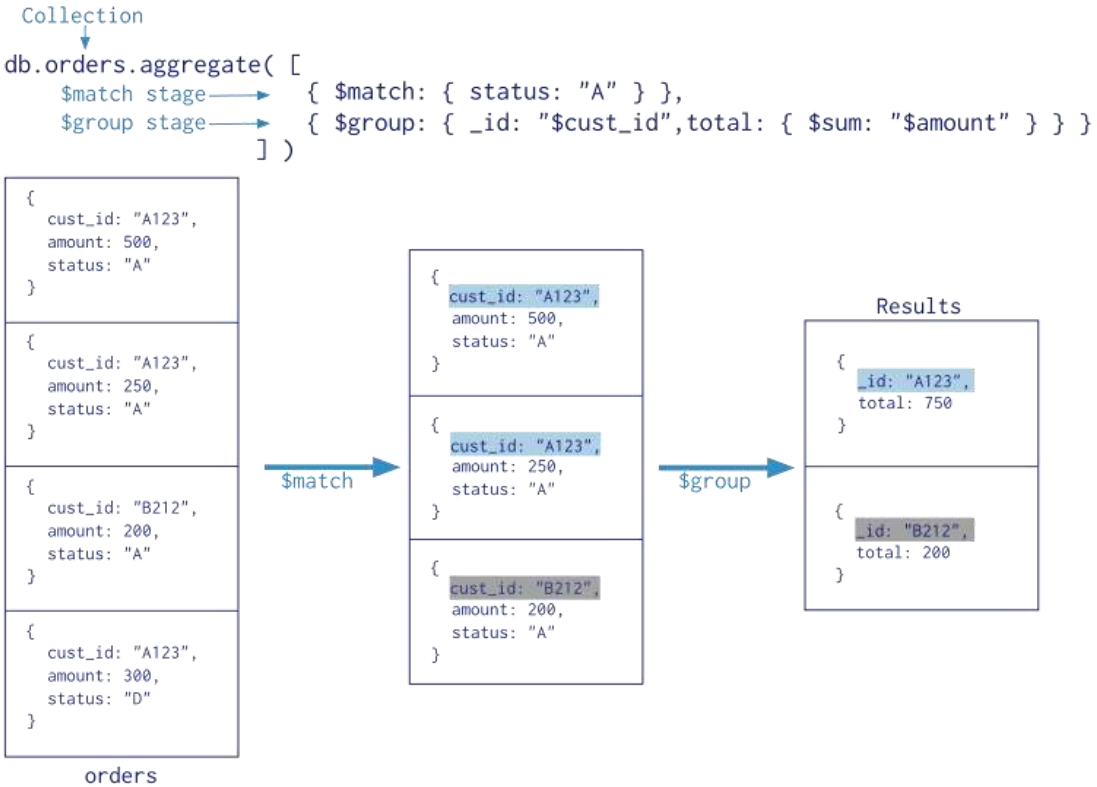
- Les opérations d'agrégation traitent des enregistrements et renvoient des résultats calculés.
- Les opérations d'agrégation regroupent des valeurs venant de plusieurs documents et peuvent effectuer une variété d'opérations sur les données groupées pour renvoyer un seul résultat.
- MongoDB propose trois méthodes d'agrégation:
  - le pipeline d'agrégation
  - la fonction MapReduce
  - des méthodes d’agrégation simple

L'Aggregation Framework

- Le pipeline d'agrégation est une suite d'étapes que les documents traversent pour être transformés en résultats agrégés.
- Les étapes peuvent être des filtres, des transformations de documents, des groupements, des tris, des calculs statistiques, …
- Le pipeline fournit une agrégation de données efficace en utilisant des opérateurs natifs. C'est la méthode préférée pour l'agrégation des données dans MongoDB.
- Le pipeline d'agrégation peut utiliser des index pour améliorer les performances au cours de certaines étapes.
- De plus, le pipeline d'agrégation dispose d'une phase d'optimisation interne.

L'Aggregation Framework

- Exemple de pipeline d'agrégation :

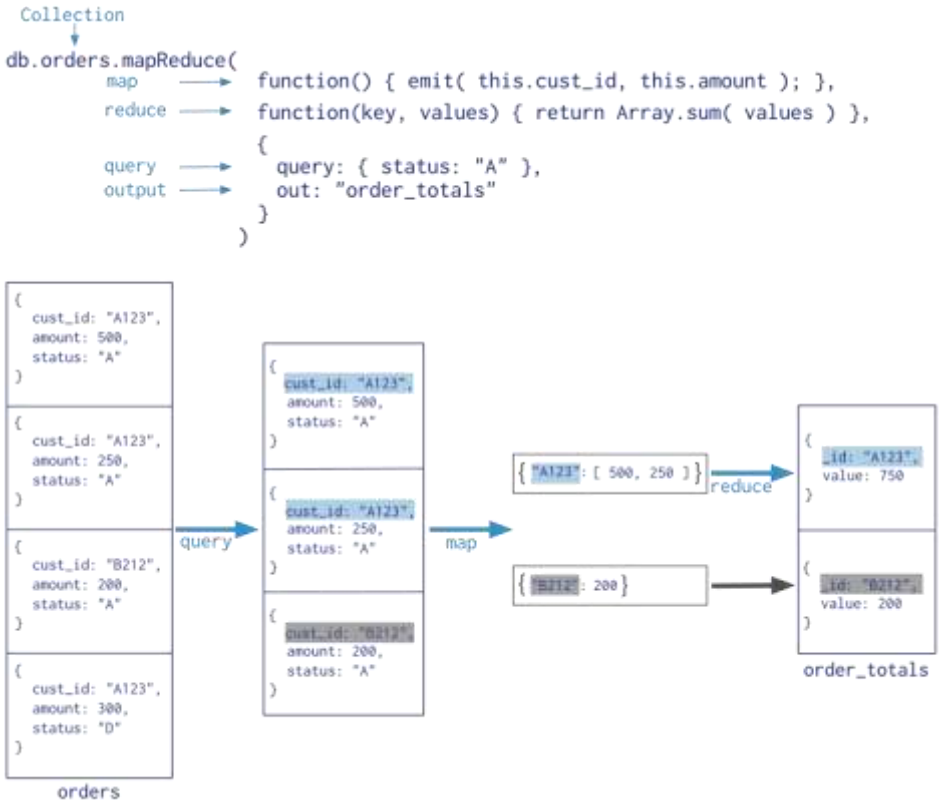


L'Aggregation Framework

- La fonction MapReduce permet de faire des agrégations (dépréciée en version 5.0).
- En général, les opérations MapReduce ont deux phases:
  - une étape « map » qui traite chaque document et émet un ou plusieurs objets pour chaque document
  - une étape « reduce » qui combine les résultats de l'étape « map »
- Optionnellement, une troisième étape « finalize » peut permettre de faire d'autres modifications après l'étape « reduce ».
- MapReduce utilise des fonctions JavaScript pour effectuer les étapes « map », « reduce » et « finalize ».
- Cela permet une plus grande flexibilité que pour le pipeline d'agrégation mais est moins efficace que ce dernier.
- Il est possible avec la fonction MapReduce de faire de l'agrégation incrémentale.

L'Aggregation Framework

- Exemple de MapReduce :



L'Aggregation Framework

- MongoDB fournit aussi 2 fonctions d'agrégation simple :
  - Pour compter le nombre de documents correspondant à un filtre de recherche :
    `db.collection.count(query, options)`
    - query : un document exprimant un filtre de recherche
    - options : un document d'options
  - Pour retourner les valeurs distinctes d'un champ :
    `db.collection.distinct(field, query, options)`
    - field : le champ à retourner
    - query : un document exprimant un filtre de recherche
    - options : un document d'options



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Requêtes avancées et agrégations

Module 7

Requêtes avancées et Agrégation

- Maîtriser les principales étapes `($match, $group, $project, $lookup, etc.)`.
- Utiliser l'API C# Aggregate() pour construire des requêtes complexes.
- Comparer l'agrégation MongoDB à une requête SQL équivalente.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Maîtriser les principales étapes de l'Agrégation

Chapitre 1

Étape $match

- Filtrage des documents

```
db.transactions.aggregate([
  { $match: { Type: "Virement", Amount: { $gte: 1000 } } }
])
```

- Sélectionne les opérations de type virement d'un montant ≥ 1000.

- Equivalent SQL :

```
SELECT * FROM transactions WHERE Type = 'Virement' AND Amount >= 1000;
```

Étape $project / $sort

- Projection de champs

```
db.transactions.aggregate([
  { $project: { _id: 0, ClientId: 1, Amount: 1, Date: 1 } }
])
```

- Ne conserve que les champs souhaités.

- Étape $sort
- Tri des résultats

```
db.transactions.aggregate([
  { $sort: { Date: -1 } }
])
```

- Trie de la plus récente à la plus ancienne.

Etape $group

- Groupement et calculs

```
db.transactions.aggregate([
  { $group: { _id: "$ClientId", total: { $sum: "$Amount" }, avg: { $avg:
"$Amount" } } }
])
```

- Calcule le total et la moyenne par élement

Étape $lookup

- Jointure entre collections

```
db.transactions.aggregate([
  { $lookup: {
      from: "clients",
      localField: "ClientId",
      foreignField: "_id",
      as: "ClientInfo"
  }}
])
```

- Simule une jointure SQL entre transactions et clients.

Étape $graphLookup

- Parcours récursif

```
db.accounts.aggregate([
  { $graphLookup: {
      from: "accounts",
      startWith: "$ParentAccount",
      connectFromField: "ParentAccount",
      connectToField: "AccountNumber",
      as: "Hierarchy"
  }}
```

- Permet de retrouver les relations hiérarchiques .

Étape $out / $merge

- Sauvegarde des résultats

```
db.transactions.aggregate([
  { $match: { Status: "Validée" } },
  { $out: "transactions_validées" }
])
```

- Sauvegarde le résultat du pipeline dans une nouvelle collection.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

l'Agrégation avec C#

Chapitre 2

Agrégation avec C# – intro

- Utilisation de Aggregate()

```
var result = await transactions.Aggregate()
    .Match(t => t.Type == "Virement")
    .Group(t => t.ClientId, g => new { Client = g.Key, Total = g.Sum(x
=> x.Amount) })
    .ToListAsync();
```

- Pipeline construit directement en C# avec `MongoDB.Driver.`

$group/$project

- `$group` – Regrouper et agréger

```
var result = await transactions.Aggregate()
    .Group(t => t.ClientId,
           g => new { Client = g.Key, Total = g.Sum(x => x.Amount) })
    .ToListAsync();
```

- Regroupe les transactions par client et calcule le total des montants.

- `$project` – Sélectionner les champs à afficher

```
var result = await transactions.Aggregate()
    .Project(t => new { t.ClientId, t.Amount, t.Date })
    .ToListAsync();
```

- Affiche uniquement certains champs utiles : identifiant, montant, date

$lookup/$out

- `$lookup` – Jointure entre collections

```
var result = await transactions.Aggregate()
    .Lookup("clients", "ClientId", "_id", "ClientInfo")
    .ToListAsync();
```

- Fait une jointure entre les collections transactions et clients
  → chaque transaction inclut les informations du client associé.
- `$out` – Sauvegarder le résultat

```
await transactions.Aggregate()
    .Match(t => t.Status == "Validée")
    .Out("transactions_validées")
    .ToListAsync();
```

- Exporte les transactions validées dans une nouvelle collection `transactions_validées`.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Comparaison entre l'Agrégation MongoDB et SQL

Chapitre 3

Comparatif SQL vs C# Aggregation

Aspect	SQL (Relationnel)	C# / MongoDB (NoSQL)
Principe	Clauses (SELECT, GROUP BY, HAVING)	Pipeline d'étapes ($match, $group, $sort, $project)
Langage	Déclaratif	Programmable / typé (LINQ ou Builder API)
Filtrage	WHERE / HAVING	$match
Regroupement	GROUP BY	$group
Tri / Limite	ORDER BY, LIMIT	$sort, $limit
Fonctions	SUM, AVG, COUNT, MAX, MIN	$sum, $avg, $count, $max, $min
Résultat	Lignes tabulaires	Documents BSON (objets C#)



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Conclusion

Conclusion

Ce que vous avez appris

- Comprendre la philosophie NoSQL et le modèle document
- Installer, configurer et manipuler MongoDB
- Réaliser des opérations CRUD via le Shell et le driver C#
- Modéliser efficacement les données avec validation et indexation
- Optimiser les requêtes avec explain et profiling

Ce que vous savez faire

- Concevoir et implémenter une application C# connectée à MongoDB
- Structurer vos modèles en embedded ou reference selon les besoins
- Gérer la performance et la scalabilité (réplication, sharding, agrégation)
- Mettre en place une architecture robuste et extensible

Merci !

CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION